ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



KỸ THUẬT DỮ LIỆU

Data Engineering — [Mã môn học]

BÁO CÁO BÀI TẬP LỚN

NỀN TẢNG KỸ THUẬT DỮ LIỆU CHO HỆ THỐNG GIÁO DỤC TRỰC TUYẾN QUY MÔ LỚN

Learning Analytics, gợi ý học tập và cảnh báo sớm nguy cơ bỏ học

Giảng viên hướng dẫn: [Họ tên giảng viên hướng dẫn]
Lớp / Nhóm: [Lớp] / [Nhóm]
Học kỳ: Học kỳ 261 (2026–2027)

Học viên thực hiện

Họ và tên	MSHV	Phần việc chính
[Họ và tên thành viên 1]	[MSHV]	[Phần việc]
[Họ và tên thành viên 2]	[MSHV]	[Phần việc]
[Họ và tên thành viên 3]	[MSHV]	[Phần việc]
[Họ và tên thành viên 4]	[MSHV]	[Phần việc]
[Họ và tên thành viên 5]	[MSHV]	[Phần việc]

[TODO: điền mã môn, lớp/nhóm, giảng viên, thành viên]

TP. Hồ Chí Minh, tháng 11 năm 2026

TÓM TẮT

Báo cáo trình bày việc thiết kế và hiện thực **DE_261**, một nền tảng kỹ thuật dữ liệu cho hệ thống giáo dục trực tuyến quy mô lớn. Nền tảng nhận bốn nguồn dữ liệu rời rạc: luồng clickstream telemetry từ web và mobile, nhật ký thay đổi giao dịch (CDC) từ PostgreSQL, kho tài liệu học tập phi cấu trúc và nhật ký thực thi bài nộp từ sandbox. Từ đó, nền tảng biến chúng thành các kho dữ liệu phục vụ tám yêu cầu nghiệp vụ (BR1–BR8) của bốn nhóm người dùng: người học, giảng viên, cố vấn học tập và quản trị viên tổ chức.

Kiến trúc dùng Apache Kafka và Debezium làm tầng tiếp nhận và event bus. Apache Spark (batch và Structured Streaming) là engine xử lý chính. Delta Lake trên object storage (MinIO) tổ chức theo mô hình medallion bronze–silver–gold, với lớp gold là một kho dữ liệu dạng lược đồ hình sao Kimball. Kho được Hive Metastore lập danh mục và Trino phục vụ truy vấn SQL. Redis là tầng phục vụ độ trễ thấp, OpenSearch phục vụ tìm kiếm lai (keyword và vector). Mọi pipeline được tổ chức thành bảy *bounded context* theo Domain-Driven Design với kiến trúc lục giác (hexagonal), nên cùng một use case chạy được trên nhiều engine và kiểm thử được mà không cần engine. Mười lăm quyết định kiến trúc được ghi lại bằng ADR, bao gồm các điểm nhóm chủ động điều chỉnh so với kiến trúc gợi ý của đề bài. **[TODO: cập nhật bảng ADR 0015]**

Về kết quả đo trên máy cá nhân với dữ liệu tổng hợp:

- **BR5**: luồng CDC đến cảnh báo bỏ học thật đi hết khoảng 10,2 giây. **[TODO: chạy lại BR5]**
- **BR8**: nội dung khoá học đã xuất bản trở nên tìm kiếm được sau trung vị 535 ms. **[TODO: chạy lại BR8]**
- **BR7, BR8**: lớp phủ thời gian thực hiển thị sau khoảng 7 giây, ổn định khi tải tăng 15 lần. **[TODO: chạy lại live overlay]**
- **BR6–BR8**: các data mart được đọc thẳng qua Trino với trung vị 42–92 ms, và qua API có cache với trung vị 1,4–2,5 ms. **[TODO: chạy lại serving]**
- **Kho dữ liệu**: toàn bộ kho (43 model) được xây trong khoảng 2–2,5 phút và vượt qua mọi cổng kiểm tra chất lượng. **[TODO: xác nhận 43 model + build time]**

Hệ thống có 346 test đơn vị **[TODO: 353 pass, 3 skip]**, các test tích hợp với dịch vụ thật, test end-to-end và test trình duyệt. Báo cáo cũng mô tả giao diện web mới cho năm nhóm người dùng, và nêu rõ những phần còn ở mức demo cùng hướng phát triển tiếp theo.

Từ khoá: kỹ thuật dữ liệu, lakehouse, Delta Lake, Kafka, Debezium, CDC, Spark Structured Streaming, kho dữ liệu Kimball, learning analytics, cảnh báo bỏ học.

MỤC LỤC

Tóm tắt **i**

Danh mục thuật ngữ và từ viết tắt **ix**

1 Bài toán, stakeholders và yêu cầu **1**

1.1 Bối cảnh 1

1.2 Phát biểu bài toán 1

1.3 Mục tiêu và phạm vi 2

1.4 Đóng góp chính 2

1.5 Phương pháp thực hiện 3

1.6 Cấu trúc báo cáo 3

1.7 Các bên liên quan và yêu cầu nghiệp vụ 3

1.8 Nguồn dữ liệu 5

1.9 Yêu cầu phi chức năng 5

1.10 Ánh xạ yêu cầu vào pipeline 6

2 Kiến trúc và lựa chọn công nghệ **8**

2.1 Tổng quan kiến trúc 8

2.2 Lựa chọn công nghệ 9

2.3 Kiến trúc phần mềm: DDD và kiến trúc lục giác 10

2.4 Ngữ nghĩa xử lý luồng 11

2.5 Các quyết định kiến trúc 11

2.6 Điều chỉnh so với kiến trúc tham chiếu của đề bài 12

3 Lakehouse và pipeline dữ liệu **14**

3.1 Kiến trúc medallion 14

3.2 Chuẩn hoá telemetry và tiến hoá schema 15

3.3 Khử định danh và phân quyền 15

3.4 Mô hình chiều Kimball ở lớp gold 16

3.4.1 Bảng chiều và khoá thay thế 17
3.5 Data mart theo yêu cầu nghiệp vụ 18
3.6 Thiết kế vật lý 19
3.7 Cổng kiểm tra chất lượng dữ liệu 19
3.8 Điều phối, dữ liệu trễ và chạy lại 20

4 Hiện thực BR1–BR8 và ứng dụng 23
4.1 BR1 — Pipeline thông báo hướng sự kiện 23
4.2 BR2 — Gợi ý khoá học và bước học tiếp theo 24
4.3 BR3 — Tiếp nhận bài nộp và phản hồi chi tiết 25
4.4 BR4 — Phát hiện câu hỏi và test case bất thường 26
4.5 BR5 — Cảnh báo sớm nguy cơ bỏ học 27
4.6 BR6 — Tiến độ kỹ năng theo tổ chức 28
4.7 BR7 — Heatmap tương tác video 28
4.8 BR8 — Đồng bộ nội dung và xu hướng tương tác 29
4.8.1 Đồng bộ cache và chỉ mục tìm kiếm 29
4.8.2 Xu hướng tương tác 29
4.9 Giao diện người dùng 30
4.9.1 Quy trình thiết kế 30
4.9.2 Hệ thống thiết kế 31
4.9.3 Kiến trúc frontend 31
4.9.4 Các màn hình chính 32
4.9.5 Kiểm thử giao diện 39

5 Kiểm thử, đánh giá và alternative solutions 41
5.1 Chiến lược kiểm thử 42
5.2 Kiểm chứng nghiệp vụ 42
5.3 Môi trường đo 44
5.4 Kết quả hiệu năng 44
5.4.1 BR4: tổng hợp SQL so với Python 44
5.4.2 BR5: độ trễ đầu cuối và thông lượng cửa sổ 45
5.4.3 BR6–BR8: phục vụ mart 45

5.4.4 BR8: từ xuất bản đến khi tìm kiếm được . . . 46
5.4.5 BR7/BR8: lớp phủ thời gian thực . . . 46
5.4.6 Thời gian build kho dữ liệu . . . 47
5.5 Thảo luận . . . 47
5.6 Alternative solutions . . . 48
5.6.1 Dữ liệu và phương pháp . . . 48
5.6.2 OLAP: PostgreSQL và ClickHouse . . . 48
5.6.3 Feature store: Redis và Memcached . . . 49
5.6.4 Streaming: Spark Structured Streaming và Flink . . . 49
5.6.5 Khi nào chuyển stack . . . 50

6 Hạn chế, kết luận và hướng phát triển 51
6.1 Hạn chế hiện tại . . . 51
6.2 Hướng phát triển . . . 52
6.3 Kết luận . . . 52

Tài liệu tham khảo 54

A Hướng dẫn chạy và tái lập kết quả 56
A.1 Yêu cầu . . . 56
A.2 Khởi động . . . 56
A.3 Kiểm thử . . . 57
A.4 Đo hiệu năng . . . 57
A.5 Đo alternative solutions . . . 57
A.6 Biên dịch báo cáo . . . 58

B Tóm tắt API 59

DANH SÁCH HÌNH VẼ

2.1 Kiến trúc tổng thể của nền tảng DE_261 8

3.1 Các lớp của lakehouse và tần suất cập nhật 14

4.1 Trích tập luật thông báo (`infra/notification/rules.yaml`) 23
4.2 Đường CDC thật của BR4 và BR5 26
4.3 Các cửa sổ đặc trưng tại mốc đánh giá T (BR5) 27
4.4 Đồng bộ nội dung BR8 dựa trên CDC 29
4.5 Mockup: trang chủ người học với thông báo (BR1) và gợi ý khoá học kèm lý do (BR2) 31
4.6 Code lab (BR3): bài nộp FizzBuzz được chấm trong sandbox, phản hồi từng test kèm diff 33
4.7 Assessment health (BR4): câu hỏi bị gắn cờ và tỷ lệ đạt theo từng test case so với ngưỡng 20% 34
4.8 Video insights (BR7): heatmap play/pause/seek của phiên bản 2 video “Joins” (390 giây), kèm các thời điểm cần xem lại 35
4.9 Learners to support (BR5): hàng đợi rủi ro và phân rã 100 điểm thành 35 + 30 + 20 + 15 36
4.10 Organisation progress (BR6): k-anonymity với $k = 5$ 37
4.11 Mockup: Content & search (BR8), theo dõi một phiên bản từ khi xuất bản đến khi tìm kiếm được 38
4.12 Pipeline health (chế độ sáng): bộ đếm của từng chặng trong luồng dữ liệu . . . 38
4.13 Trang người học trên màn hình điện thoại (390 px): thanh bên chuyển thành tab ngang 39

5.1 BR4: thời gian tổng hợp và truy vấn theo quy mô (trung bình 3 lần, trục log) . . 44
5.2 Độ trễ phục vụ các mart BR6–BR8 (30 request mỗi trường hợp) **[TODO: chạy lại serving]** 45
5.3 BR8: độ trễ từ xuất bản đến khi tìm kiếm được, 20 lần xuất bản liên tiếp **[TODO: chạy lại content sync]** 46

5.4 Thời gian build toàn bộ 43 model theo khối lượng dữ liệu (trung bình 2 lần) **[TODO: chạy lại 35/70/140 ngày]** 47

DANH SÁCH BẢNG

1.1 Yêu cầu nghiệp vụ và nhu cầu kỹ thuật dữ liệu 4
1.2 Bốn nguồn dữ liệu và cách tiếp nhận 5
1.3 Ánh xạ yêu cầu nghiệp vụ vào pipeline và runtime 6

2.1 Công nghệ theo vai trò 9
2.2 Các lớp trong một bounded context 10
2.3 Tóm tắt các quyết định kiến trúc (ADR) 11

3.1 Hợp đồng dữ liệu của từng lớp 14
3.2 Phân quyền truy cập kho dữ liệu qua Trino 16
3.3 Ma trận bus của lớp gold (D: vai trò ngày, •: khoá chiều) 17
3.4 Các data mart và kho đặc trưng 18
3.5 Các cổng kiểm tra chất lượng 20
3.6 Các DAG Airflow 21
3.7 Cơ chế xử lý dữ liệu trễ, trùng lặp và sai thứ tự 21

4.1 Các tầng của luật gợi ý (BR2) 25
4.2 Quy tắc phát hiện bất thường của BR4 (cấu hình được) 26
4.3 Các tín hiệu rủi ro và điểm (BR5) 27
4.4 Trọng số hoạt động trong điểm tương tác (`EngagementWeights`) 30
4.5 Workspace, đường dẫn và trang 32

5.1 Các tầng kiểm thử 42
5.2 Các kịch bản kiểm chứng nghiệp vụ và kết quả 43
5.3 Môi trường đo hiệu năng 44
5.4 BR5: chặng Kafka → Spark → cửa sổ đã lưu (trung bình 3 lần) **[TODO: số mới]** 45
5.5 Độ trễ từ sự kiện đến khi hiển thị trong API (3 đợt mỗi kích thước) **[TODO: chạy lại live overlay]** 46
5.6 Dataset 5 GiB dùng cho benchmark alternative 48
5.7 OLAP: load và query p50 khi số dòng tăng 48

5.8 Feature store: Redis và Memcached, cùng batch 1.000 key 49
5.9 Streaming filter: cùng input, cùng output 50
5.10 Điều kiện chuyển sang alternative 50

B.1 Các endpoint chính 59

DANH MỤC THUẬT NGỮ VÀ TỪ VIẾT TẮT

Thuật ngữ	Giải thích
ADR	*Architecture Decision Record*: tài liệu ghi lại một quyết định kiến trúc, bối cảnh, phương án bị loại và hệ quả.
BR	*Business Requirement*: yêu cầu nghiệp vụ (BR1–BR8) trong đề bài.
Bronze / Silver / Gold	Ba lớp của kiến trúc medallion: dữ liệu thô, dữ liệu đã làm sạch và khử định danh, dữ liệu mô hình hoá cho phân tích.
CDC	*Change Data Capture*: bắt các thay đổi cấp dòng từ nhật ký giao dịch (WAL) của cơ sở dữ liệu.
CDF	*Change Data Feed*: tính năng của Delta Lake ghi lại các thay đổi cấp dòng của bảng.
DDD	*Domain-Driven Design*: thiết kế hướng miền nghiệp vụ, chia hệ thống thành các *bounded context*.
DLQ	*Dead-Letter Queue*: nơi lưu sự kiện bị từ chối kèm lý do để xử lý lại.
Event time	Thời điểm sự kiện xảy ra tại nguồn, khác với thời điểm hệ thống nhận được.
Exactly-once	Đảm bảo mỗi bản ghi tác động lên kết quả đúng một lần dù có lỗi hoặc gửi lại.
Hexagonal	Kiến trúc lục giác (ports and adapters): lõi nghiệp vụ chỉ phụ thuộc vào các cổng trừu tượng.
HMS	Hive Metastore: dịch vụ danh mục (catalog) chung cho các bảng Delta.
Idempotent	Thực thi lặp lại nhiều lần cho cùng kết quả như thực thi một lần.
k-anonymity	Nguyên tắc không công bố số liệu của nhóm có ít hơn k cá thể; ở đây $k = 5$.
LSN	*Log Sequence Number*: vị trí trong WAL của PostgreSQL, dùng để sắp thứ tự thay đổi.
Mart	Bảng tổng hợp phục vụ một yêu cầu nghiệp vụ cụ thể.
Medallion	Mô hình tổ chức lakehouse theo các lớp chất lượng tăng dần.
OLTP	*Online Transaction Processing*: cơ sở dữ liệu giao dịch của hệ thống quản lý học tập (LMS).
PII	*Personally Identifiable Information*: thông tin định danh cá nhân.

Thuật ngữ	Giải thích
Pseudonym	Bút danh: giá trị băm có khoá (HMAC) thay cho định danh thật.
SCD	*Slowly Changing Dimension*: chiến lược lưu lịch sử thay đổi của bảng chiều (loại 1 ghi đè, loại 2 lưu phiên bản).
Watermark	Ngưỡng thời gian sự kiện mà engine streaming coi là đã nhận đủ dữ liệu trước đó.

CHƯƠNG 1

BÀI TOÁN, STAKEHOLDERS VÀ YÊU CẦU

Phần này trình bày bối cảnh, các bên liên quan, yêu cầu nghiệp vụ và nguồn dữ liệu của nền tảng.

1.1 Bối cảnh

Các nền tảng giáo dục trực tuyến quy mô lớn sinh ra dữ liệu liên tục và đa dạng. Mỗi lần người học mở bài giảng, tua video, nộp bài lập trình hay thảo luận đều để lại dấu vết. Hệ thống quản lý học tập (LMS) ghi nhận tiến độ, bài nộp và điểm số. Kho học liệu gồm transcript video, slide và tài liệu PDF được cập nhật theo từng phiên bản. Các sandbox chấm bài sinh ra nhật ký thực thi chi tiết. Những dữ liệu này có giá trị lớn cho *learning analytics*: giúp giảng viên biết bài tập nào có vấn đề, giúp cố vấn học tập phát hiện sớm người học có nguy cơ bỏ dở, và giúp người học nhận phản hồi và gợi ý kịp thời.

Tuy vậy, các nguồn này rời rạc, khác nhau về định dạng, tốc độ và độ tin cậy. Telemetry từ thiết bị di động có thể đến trễ nhiều ngày hoặc sai thứ tự. Thay đổi trong cơ sở dữ liệu giao dịch phải được bắt theo đúng thứ tự mà không làm nặng hệ thống production. Tài liệu học tập cần được chia đoạn và quản lý phiên bản. Dữ liệu cá nhân của người học cần được bảo vệ, nhất là khi báo cáo cho các tổ chức đối tác (B2B). Đây là bài toán kỹ thuật dữ liệu điển hình, đòi hỏi kết hợp xử lý luồng, xử lý theo lô, lưu trữ phân tích và phục vụ độ trễ thấp [1].

1.2 Phát biểu bài toán

Đề bài DE_261 yêu cầu xây dựng một nền tảng dữ liệu biến bốn nguồn dữ liệu thô thành các kho dữ liệu phục vụ ba mục tiêu: phân tích học tập, hệ thống gợi ý thích ứng và phát hiện sớm nguy cơ bỏ học. Đề bài xác định tám yêu cầu nghiệp vụ (BR1–BR8) cho bốn nhóm người dùng (người học, giảng viên, cố vấn học tập, quản trị viên tổ chức). Đề bài cũng gợi ý một kiến trúc tham chiếu gồm Kafka và Debezium cho tiếp nhận, Flink cho xử lý luồng, Spark cho xử lý lô, Delta Lake hoặc Iceberg cho lakehouse, và Redis hoặc Cassandra cho tầng phục vụ. Phần này phân tích chi tiết các yêu cầu này.

1.3 Mục tiêu và phạm vi

Nhóm đặt ra các mục tiêu sau:

1. Thiết kế một kiến trúc dữ liệu đầu cuối bao phủ cả tám yêu cầu nghiệp vụ, bám sát kiến trúc tham chiếu. Mọi điểm điều chỉnh đều có lý do và được ghi lại.
2. Hiện thực các pipeline chạy được trên máy cá nhân bằng Docker Compose: tiếp nhận CDC thật qua Debezium, lakehouse Delta Lake với kho dữ liệu dạng hình sao, các data mart theo từng BR, và các đường xử lý thời gian thực.
3. Bảo đảm các tính chất quan trọng của hệ thống dữ liệu: xử lý theo thời gian sự kiện, chịu được dữ liệu trễ và trùng lặp, khả năng chạy lại idempotent, khử định danh và k-anonymity, kiểm tra chất lượng dữ liệu.
4. Cung cấp API và giao diện web cho từng nhóm người dùng, để kết quả của pipeline thực sự được sử dụng.
5. Đánh giá tính đúng đắn và hiệu năng bằng kiểm thử tự động và đo đạc có thể tái lập.

Về phạm vi: toàn bộ dữ liệu dùng trong báo cáo là **dữ liệu tổng hợp** (synthetic). Hệ thống được triển khai trên một máy, không nhằm chứng minh khả năng mở rộng ngang hay độ sẵn sàng cao. Các số đo hiệu năng là quan sát cục bộ, không phải cam kết năng lực.

1.4 Đóng góp chính

- Một **lakehouse Delta Lake** ba lớp với 50 bảng và 43 model SQL. **[TODO: xác nhận 50 bảng / 43 model / 9 dim + 11 fact]** Lớp gold là kho dữ liệu Kimball gồm 9 bảng chiều và 11 bảng sự kiện, dùng khoá thay thế tất định. Các mart cho BR4, BR6, BR7, BR8 và một kho đặc trưng ngoại tuyến cho BR2. Kho được lập danh mục bằng Hive Metastore và phục vụ qua Trino có phân quyền theo vai trò.
- Một đường **CDC thật**: PostgreSQL WAL, rồi Debezium, Kafka, Spark Structured Streaming, đến các mart, trạng thái rủi ro và cảnh báo cho BR4/BR5. Đường này chịu được phát lại (replay), cập nhật và xoá, và khởi động lại checkpoint.
- Đồng bộ nội dung **BR8 dựa trên CDC**: vô hiệu hoá cache Redis, chia đoạn và nhúng vector vào OpenSearch, tìm kiếm lai, với ngữ nghĩa idempotent theo phiên bản.
- **Lớp phủ thời gian thực** cho heatmap video (BR7) và điểm tương tác (BR8) trên Redis, ghép với mart theo ngày.
- Mười bốn **ADR**, một kiến trúc phần mềm DDD/hexagonal có test bảo vệ ranh giới, và một bộ kiểm thử nhiều tầng có kèm bằng chứng đo đạc.
- Một **giao diện web** mới cho năm không gian làm việc (workspace), thiết kế từ mockup và

nối với API thật.

1.5 Phương pháp thực hiện

Nhóm làm việc theo vòng lặp ngắn: phân tích yêu cầu, ghi quyết định kiến trúc (ADR), hiện thực, rồi kiểm chứng bằng test và số đo. Lõi nghiệp vụ (các *policy* và *use case*) được viết trước, kiểm thử độc lập với engine. Sau đó các adapter cho Kafka, Spark, Delta, Redis, PostgreSQL và OpenSearch được bổ sung dần. Mỗi nhóm BR có một tài liệu kế hoạch và một tài liệu hiện thực trong thư mục `docs/`. Các tài liệu này ghi rõ cái gì đã chạy thật, cái gì còn là demo, và các lỗi tìm ra trong quá trình kiểm chứng.

1.6 Cấu trúc báo cáo

Báo cáo được tổ chức thành sáu phần:

- **Phần 1** — bài toán, stakeholders, tám yêu cầu nghiệp vụ, nguồn dữ liệu và yêu cầu phi chức năng.
- **Phần 2** — kiến trúc tổng thể, lựa chọn công nghệ, kiến trúc phần mềm và các quyết định kiến trúc.
- **Phần 3** — lakehouse, mô hình medallion, khử định danh, kho dữ liệu Kimball, data mart và kiểm tra chất lượng.
- **Phần 4** — thiết kế và hiện thực BR1–BR8, giao diện người dùng và cách chạy demo/service.
- **Phần 5** — kiểm thử, đánh giá hiệu năng và mục alternative solutions để điền sau.
- **Phần 6** — hạn chế, hướng phát triển và kết luận.

1.7 Các bên liên quan và yêu cầu nghiệp vụ

Đề bài xác định bốn nhóm người dùng và tám yêu cầu nghiệp vụ. Bảng 1.1 tóm tắt mỗi yêu cầu cùng nhu cầu kỹ thuật dữ liệu tương ứng, theo cách đọc của nhóm.

Bảng 1.1. Yêu cầu nghiệp vụ và nhu cầu kỹ thuật dữ liệu

Mã	Người dùng	Yêu cầu nghiệp vụ	Nhu cầu kỹ thuật dữ liệu
BR1	Người học	Nhận thông báo khi có bài tập mới, deadline sắp đến hoặc nội dung được cập nhật	Pipeline thông báo hướng sự kiện: nhận sự kiện LMS qua message broker, lọc bằng rule engine, gửi theo thời gian thực
BR2	Người học	Nhận gợi ý khoá học và bước học tiếp theo	Pipeline đặc trưng hai nhánh: đặc trưng theo lô (Spark) và đặc trưng phiên (streaming) qua feature store
BR3	Người học	Nhận phản hồi chi tiết khi nộp bài lập trình/SQL	Tiếp nhận bài nộp tách rời qua hàng đợi đệm; chuẩn hoá kết quả kiểm thử (stderr, diff test case, thời gian chạy)
BR4	Giảng viên	Phát hiện câu hỏi hoặc test case có tỷ lệ trượt bất thường	Aggregation mart: tỷ lệ đạt theo từng test case và câu hỏi để lọc đề lỗi logic hoặc quá khó
BR5	Cố vấn học tập	Nhận cảnh báo sớm học viên có nguy cơ bỏ dở	CDC và cửa sổ trượt: đọc log thay đổi từ OLTP, tính điểm rủi ro, phát sinh cảnh báo
BR6	Quản trị viên (B2B)	Theo dõi tiến độ kỹ năng và tỷ lệ hoàn thành theo tổ chức	Pipeline medallion: chuẩn hoá telemetry có phiên bản schema, khử định danh, báo cáo tổ chức với ngưỡng nhóm tối thiểu
BR7	Giảng viên	Xem heatmap tương tác video bài giảng	Tổng hợp chi tiết: mật độ play/pause/tua theo từng giây video
BR8	Giảng viên	Đồng bộ trạng thái, vô hiệu hoá cache, làm mới chỉ mục tìm kiếm khi nội dung thay đổi	Pipeline xu hướng tương tác: tổng hợp hoạt động theo cửa sổ thời gian, tính điểm tương tác, phát hiện xu hướng giảm

Nhận xét về BR8

Cột yêu cầu nghiệp vụ của BR8 (đồng bộ cache và chỉ mục tìm kiếm khi nội dung thay đổi) và cột nhu cầu kỹ thuật (pipeline xu hướng tương tác) mô tả hai nhu cầu khác nhau. Nhóm hiện thực **cả hai**: đồng bộ nội dung dựa trên CDC (Mục 4.8.1) và mart xu hướng tương tác (Mục 4.8.2).

1.8 Nguồn dữ liệu

Bảng 1.2 mô tả bốn nguồn dữ liệu thô, đặc tính của chúng và cách nền tảng tiếp nhận. Nguồn nào cũng có một thách thức riêng, và thách thức đó quyết định thiết kế của tầng tiếp nhận.

Bảng 1.2. Bốn nguồn dữ liệu và cách tiếp nhận

Nguồn	Đặc tính	Thách thức	Tiếp nhận
Telemetry (web, Android, iOS)	JSON bán cấu trúc kiểu xAPI [2], lưu lượng lớn	Đến trễ, sai thứ tự (thiết bị offline), gửi lại, nhiều phiên bản schema, chứa PII, đồng hồ thiết bị lệch	Topic `telemetry.events.raw.v1`, khoá theo người học; DLQ `telemetry.events.dlq.v1`
CDC từ PostgreSQL	Thay đổi cấp dòng từ WAL: tiến độ, bài nộp, điểm	Giữ thứ tự theo khoá, không ảnh hưởng truy vấn production, xử lý cập nhật và xoá	Debezium (plugin `pgoutput`, replication slot) vào 11 topic `cdc.lms.public.*`
Kho học liệu (object storage)	Transcript, slide, PDF; văn bản dài	Chia đoạn theo ngữ nghĩa, quản lý phiên bản nghiêm ngặt	Đối tượng bất biến theo phiên bản trong MinIO; thay đổi đi qua CDC của bảng `course_content`
Nhật ký sandbox	stdout, stderr, thời gian chạy, bộ nhớ, trạng thái assertion	Chuẩn hoá đầu ra, tách lỗi hạ tầng khỏi lỗi của người học	Topic `grading.submission_received.v1`, `grading.sandbox_result.raw.v1`, `grading.submission_graded.v1`

Danh mục topic Kafka gồm 26 topic, được khai báo dưới dạng mã trong `infra/kafka/topics.yaml`. Ngoài các topic ở trên còn có các topic sự kiện miền: `lms.*` (bài tập mới, sắp đến hạn, khoá học cập nhật), `risk.*` (điểm rủi ro, cảnh báo), `notification.*` và `content.*`.

1.9 Yêu cầu phi chức năng

Từ đặc tính của nguồn dữ liệu và nhu cầu của người dùng, nhóm rút ra các yêu cầu phi chức năng sau. Chúng là tiêu chí để đánh giá các quyết định kiến trúc ở Chương 2.

Độ trễ phù hợp từng BR

Thông báo, phản hồi chấm bài và cảnh báo rủi ro cần độ trễ tính bằng giây. Báo cáo tổ chức và heatmap chấp nhận cập nhật theo ngày, kèm lớp phủ cho hôm nay. Tra cứu đặc trưng cho gợi ý cần dưới 10 ms.

Đúng một lần, idempotent

Gửi lại từ Kafka, chạy lại job và backfill không được làm sai số liệu. Mọi lệnh batch phải idempotent theo ngày chạy.

Xử lý theo thời gian sự kiện

Cửa sổ thời gian tính theo lúc sự kiện xảy ra, có watermark. Dữ liệu trễ vẫn được lưu và gắn cờ, không bị bỏ qua âm thầm.

Quyền riêng tư

Định danh thật chỉ tồn tại ở lớp thô và bị hạn chế truy cập. Các lớp phân tích chỉ dùng bút danh. Báo cáo tổ chức không công bố số liệu của nhóm dưới $k = 5$ người.

Tiến hoá schema

Payload của client có nhiều phiên bản; hệ thống phải chuyển đổi về phiên bản hiện hành hoặc từ chối có lý do.

Tái lập và kiểm toán

Có thể dựng lại toàn bộ dữ liệu từ lớp thô, xem lại trạng thái cũ (time travel), và truy vết mỗi lần build.

Chất lượng dữ liệu

Có cổng kiểm tra sau mỗi lớp; lỗi nghiêm trọng phải dừng pipeline.

Chạy được trên máy cá nhân

Toàn bộ hệ thống chạy bằng Docker Compose, chỉ dùng CPU, không phụ thuộc dịch vụ trả phí.

1.10 Ánh xạ yêu cầu vào pipeline

Bảng 1.3 cho thấy mỗi BR được giao cho bounded context nào, chạy trên runtime nào và được phục vụ từ đâu. Cột cuối nêu mức độ hoàn thiện thực tế; Chương 6 giải thích thêm.

Bảng 1.3. Ánh xạ yêu cầu nghiệp vụ vào pipeline và runtime

BR	Context	Runtime xử lý	Lưu trữ / phục vụ	Mức hoàn thiện
1	notification	Rule engine trên sự kiện Kafka	Redis (dedup, giới hạn tần suất)	Logic hoàn chỉnh; runtime demo

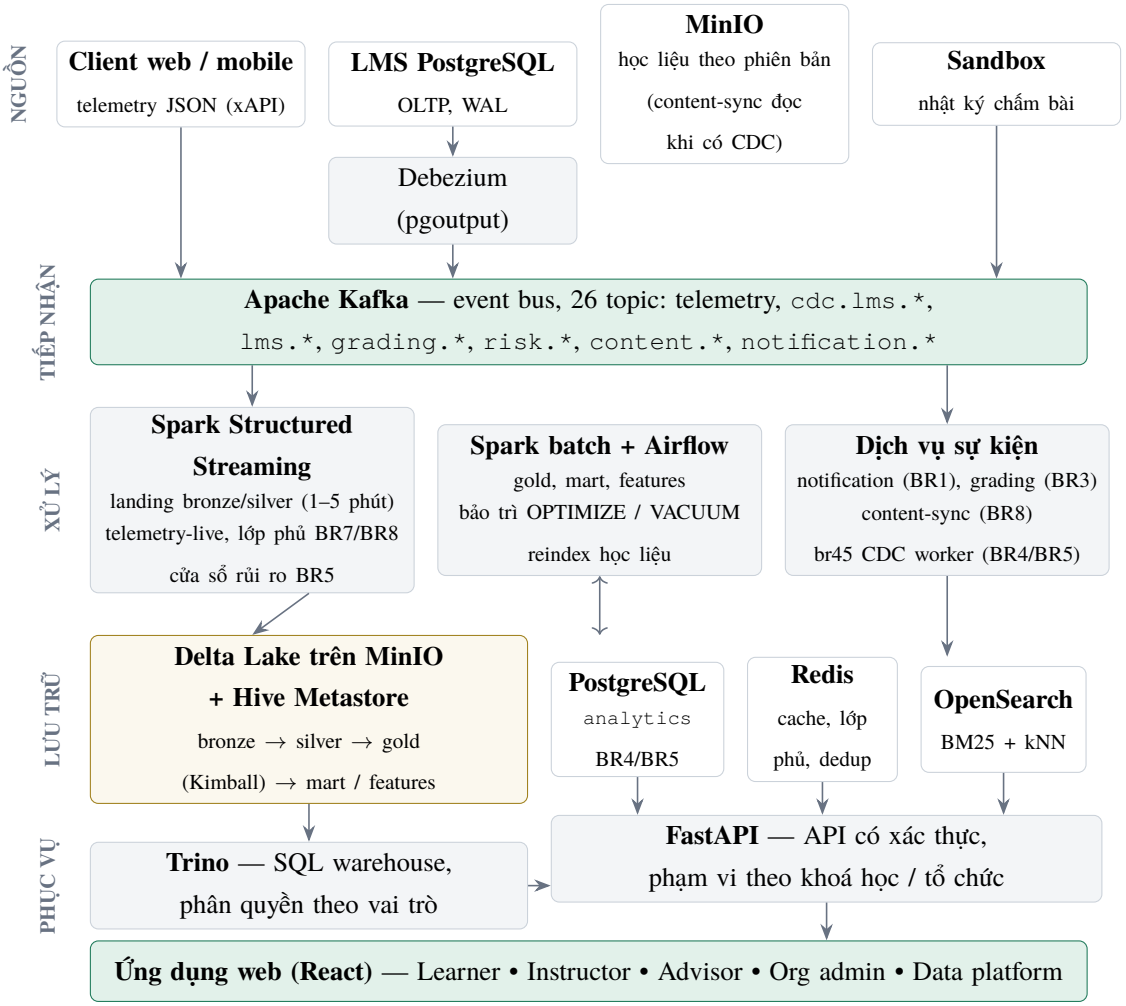
BR	Context	Runtime xử lý	Lưu trữ / phục vụ	Mức hoàn thiện
2	recommendation	Luật tại thời điểm truy vấn; đặc trưng lô `features.learner_longterm`	Delta (ngoại tuyến), API	Rút gọn về rule-based
3	grading	Hàng đợi Kafka, sandbox, `FeedbackPolicy`	API	Logic hoàn chỉnh; runtime demo
4	analytics + br45	CDC, SQL aggregation	PostgreSQL `analytics`; lịch sử ở `mart.assessment_item_stats`	Chạy thật
5	risk + br45	CDC, Spark Structured Streaming (cửa sổ 7 ngày)	PostgreSQL; lịch sử ở `gold.fct_risk_score_daily`	Chạy thật
6	telemetry + analytics	Medallion Spark	Delta, Trino (`mart.org_progress`)	Chạy thật
7	analytics	Spark batch + lớp phủ streaming	Delta, Trino, Redis	Chạy thật
8	content + analytics	CDC, dịch vụ content-sync; Spark batch + lớp phủ	Redis, OpenSearch, Delta, Trino	Chạy thật

CHƯƠNG 2

KIẾN TRÚC VÀ LỰA CHỌN CÔNG NGHỆ

2.1 Tổng quan kiến trúc

Hình 2.1 trình bày kiến trúc đầu cuối của nền tảng, theo sáu tầng từ trên xuống. Tầng nguồn gồm bốn nguồn dữ liệu. Tầng tiếp nhận dùng Kafka làm event bus duy nhất, với Debezium bắt thay đổi từ WAL của PostgreSQL. Tầng xử lý dùng Spark cho cả luồng và lô, cùng các dịch vụ tiêu thụ sự kiện. Tầng lưu trữ gồm lakehouse Delta Lake, PostgreSQL cho các mart thời gian thực của BR4/BR5, Redis và OpenSearch. Tầng phục vụ gồm Trino (SQL) và một API FastAPI. Trên cùng là ứng dụng web cho năm nhóm người dùng.



Hình 2.1. Kiến trúc tổng thể của nền tảng DE_261

Ba nguyên tắc xuyên suốt kiến trúc:

- **Kafka là ranh giới giữa các context.** Các bounded context chỉ giao tiếp qua topic Kafka và bảng lakehouse, không dùng chung cơ sở dữ liệu. Nhờ đó mỗi context có thể triển khai và thay đổi độc lập.
- **Spark là bên ghi duy nhất của lakehouse** (ADR 0011). Các job thời gian thực chỉ ghi ra Kafka và Redis. Điều này tránh xung đột commit khi nhiều engine cùng ghi một bảng Delta, và giữ logic idempotent ở một nơi.
- **Lakehouse là nguồn sự thật cho phân tích.** Redis và các mart phục vụ đều dựng lại được từ lakehouse hoặc từ dữ liệu thô (ADR 0006).

2.2 Lựa chọn công nghệ

Bảng 2.1 liệt kê công nghệ cho từng vai trò. Các thành phần chính gồm Kafka [3], Debezium [4], Spark [5] với Structured Streaming [6], Delta Lake [7], Trino (hậu duệ của Presto [8]), Redis [9] và OpenSearch [10]. Nhóm giữ các lựa chọn của đề bài khi chúng chạy được trên máy cá nhân. Khi phải thay thế, nhóm chọn thành phần đóng cùng vai trò và ghi lại bằng ADR.

Bảng 2.1. Công nghệ theo vai trò

Vai trò	Công nghệ	Lý do chọn
Tiếp nhận, event bus	Apache Kafka	Bộ đệm cho telemetry tốc độ cao, giữ thứ tự theo khoá, phát lại được; dùng chung làm event bus cho các sự kiện LMS (ADR 0003)
CDC	Debezium (pgoutput)	Đọc replication slot nên không tạo tải truy vấn lên production; giữ LSN để sắp thứ tự
Xử lý luồng	Spark Structured Streaming	Watermark theo event time, checkpoint, sink giao dịch Delta; dùng chung engine với batch (ADR 0009, 0014)
Xử lý lô	Apache Spark 3.5	Hệ sinh thái Delta, xử lý song song cho model SQL và chia đoạn học liệu (ADR 0004)
Lakehouse	Delta Lake 3.3 trên MinIO	ACID MERGE, time travel, Change Data Feed; tương thích Databricks (ADR 0005)
Catalog	Hive Metastore 3.1.3	Danh mục chung cho Spark và Trino; bản 3.1 tương thích Spark 3.5 (ADR 0010)
SQL warehouse	Trino	Truy vấn trực tiếp bảng Delta, ANSI SQL, phân quyền theo schema, không cần bản sao thứ hai (ADR 0010)

Vai trò	Công nghệ	Lý do chọn
Phục vụ độ trễ thấp	Redis	Tra cứu theo khoá dưới mili-giây, TTL; dùng cho cache, lớp phủ thời gian thực, dedup thông báo (ADR 0006)
Mart thời gian thực BR4/BR5	PostgreSQL 16	Phục vụ giao dịch, cập nhật đúng ngay khi nguồn sửa (ADR 0009)
Tìm kiếm	OpenSearch 2.17	BM25 kết hợp vector HNSW; xếp hạng lai bằng reciprocal rank fusion (ADR 0013)
Điều phối	Apache Airflow	DAG theo lịch và theo dataset; mọi task idempotent theo ngày chạy
API, giao diện	FastAPI, React 18 + TypeScript	Một API chung cho mọi BR; giao diện không dùng thư viện UI hay biểu đồ ngoài

2.3 Kiến trúc phần mềm: DDD và kiến trúc lục giác

Toàn bộ mã nằm trong một gói Python `edu_platform` (monorepo, ADR 0002), chia thành bảy bounded context theo Domain-Driven Design [11]: `telemetry`, `notification`, `recommendation`, `grading`, `risk`, `analytics` và `content`. Mỗi context có bốn lớp theo kiến trúc lục giác [12] như Bảng 2.2. Một test kiến trúc (`tests/unit/test_architecture.py`) kiểm tra quy tắc phụ thuộc giữa các lớp.

Bảng 2.2. Các lớp trong một bounded context

Lớp	Chứa gì	Được phụ thuộc vào
`domain/`	Thực thể, value object, aggregate, sự kiện miền, *policy* (ví dụ `EngagementPolicy`, `MinCohortPolicy`)	Chỉ `shared_kernel.domain`
`application/`	Use case (mỗi lớp một hàm `execute`), các cổng `Protocol`	domain
`infrastructure/`	Adapter: Kafka, Redis, Delta, S3, Trino, OpenSearch, YAML	Mọi thứ
`entrypoints/`	Điểm ghép nối cho job Spark, dịch vụ, API	Mọi thứ

Lợi ích lớn nhất của cách tổ chức này là **một logic, nhiều engine**. Ví dụ, use case `NormalizeEvents` (kiểm tra hợp lệ, chuyển schema, khử định danh, gắn cờ trễ) chạy trong job streaming thời gian thực, trong job silver của lakehouse, và trong runtime demo. Kết quả luôn giống nhau,

kể cả bút danh và quyết định đưa vào DLQ. Các ngưỡng nghiệp vụ dùng trong SQL của mart cũng được lấy từ value object của domain, và có test đơn vị giữ hai phía đồng bộ.

2.4 Ngữ nghĩa xử lý luồng

- **Thời gian sự kiện ở mọi nơi** theo mô hình Dataflow [13]. Watermark cho phép sự kiện đến muộn tối đa 5 phút với telemetry và 10 giây với CDC. Riêng cửa sổ rủi ro BR5 dùng watermark 21 ngày, vì dữ liệu học tập có thể được đồng bộ muộn.
- **Dữ liệu trễ không bị mất.** Sự kiện trễ hơn watermark vẫn được lưu và gắn cờ. Các model gold tự mở rộng cửa sổ tính toán về ngày của sự kiện trễ nhất (cơ chế `late_data`, Mục 3.8).
- **Đúng một lần.** Landing dùng checkpoint của Structured Streaming kết hợp sink giao dịch Delta. Lớp silver dùng MERGE idempotent theo khoá nghiệp vụ. Lớp phủ Redis ghi giá trị tuyệt đối thay vì cộng dồn, nên phát lại không đếm hai lần.
- **Thứ tự CDC.** Thay đổi được sắp theo `source.lsn` cho từng khoá. Thay đổi có LSN cũ hơn không thể ghi đè bản ghi mới hơn.
- **Đồng hồ thiết bị lệch.** Sự kiện có thời gian sau thời điểm tiếp nhận quá 2 phút được ghim về thời điểm tiếp nhận (`ClockSkewPolicy`). Nếu không, một thiết bị có đồng hồ chạy nhanh sẽ đẩy watermark lên và làm rơi sự kiện đúng hạn của mọi người khác.

2.5 Các quyết định kiến trúc

Nhóm ghi mỗi quyết định quan trọng thành một ADR [14] trong `docs/adr/`. ADR đã chấp nhận thì không sửa; quyết định mới thay thế quyết định cũ bằng một ADR mới. Bảng 2.3 tóm tắt mười lăm ADR. **[TODO: thêm ADR 0015]**

Bảng 2.3. Tóm tắt các quyết định kiến trúc (ADR)

ADR	Quyết định	Lý do và hệ quả chính
0001	Ghi quyết định bằng ADR	Tránh tranh luận lại; lưu lý do, không chỉ kết quả
0002	Monorepo Python, DDD + hexagonal	Use case độc lập engine; một chi phí nhỏ cho các lớp gián tiếp
0003	Kafka + Debezium cho tiếp nhận và CDC	Giữ thứ tự theo thực thể; phải giám sát replication slot
0004	Flink cho luồng, Spark cho lô	Lựa chọn ban đầu theo đề bài; được điều chỉnh bởi ADR 0009 và 0014

ADR	Quyết định	Lý do và hệ quả chính
0005	Lakehouse Delta Lake medallion	ACID upsert, time travel, ranh giới PII; cần job bảo trì chống file nhỏ
0006	Redis cho tầng phục vụ	Dưới mili-giây, TTL; Redis không phải nguồn sự thật
0007	Khử định danh ở ranh giới silver	HMAC-SHA256 có phiên bản khoá; báo cáo tổ chức áp dụng $k = 5$
0008	Runtime demo trong tiến trình	Trình diễn đầu cuối trên laptop bằng adapter trong bộ nhớ, cùng các cổng như production
0009	PostgreSQL mart + Spark Structured Streaming cho BR4/BR5	Một driver Spark thay vì hoàn thiện cả hai engine; độ trễ CDC đến cảnh báo khoảng 10 giây
0010	Trino + Hive Metastore làm SQL warehouse	Đóng vai "Databricks SQL + catalog" trên chính các bảng Delta, không có bản sao thứ hai
0011	Spark là bên ghi duy nhất của Delta	Connector Flink cho Delta đã lỗi thời; tránh xung đột commit
0012	Gold là lược đồ hình sao Kimball, khoá tất định	`xxhash64` của khoá tự nhiên; dựng lại tái tạo đúng khoá; thành viên suy luận cho chiều đến muộn
0013	Đồng bộ nội dung BR8 bằng CDC vào OpenSearch	Một đường đồng bộ cho mọi bên ghi; idempotent theo phiên bản đã chỉ mục
0014	Spark Structured Streaming cho telemetry nhanh và lớp phủ BR7/BR8	Các job Flink chưa tồn tại; độ trễ đo được khoảng 7 giây đủ cho lớp phủ
0015	Session features trên Spark Structured Streaming + Redis state	Spark 3.5 không hỗ trợ session window ở update mode; state nhỏ đặt trong Redis để giữ freshness 30 giây

2.6 Điều chỉnh so với kiến trúc tham chiếu của đề bài

Nhóm giữ nguyên các vai trò trong kiến trúc của đề bài, nhưng có bốn điểm điều chỉnh có chủ đích:

1. **Spark Structured Streaming thay Flink cho BR5 và các lớp phủ BR7/BR8** (ADR 0009, 0014). Đề bài đề xuất Flink [15] cho xử lý luồng có trạng thái. Khi hiện thực, các job Flink chỉ là stub, trong khi Spark Structured Streaming đã chạy cho landing lakehouse. Dùng một engine giúp giảm hạ tầng phải vận hành, và cho phép chạy lại đúng code chuẩn

hoá của lớp silver. Đổi lại, độ trễ khoảng 7–10 giây, cao hơn mức dưới một giây mà Flink có thể đạt. Với mục đích cảnh báo cho cố vấn và lớp phủ trên mart theo ngày, mức này là chấp nhận được. Các dịch vụ Flink vẫn có trong Compose để chuyển đổi khi cần.

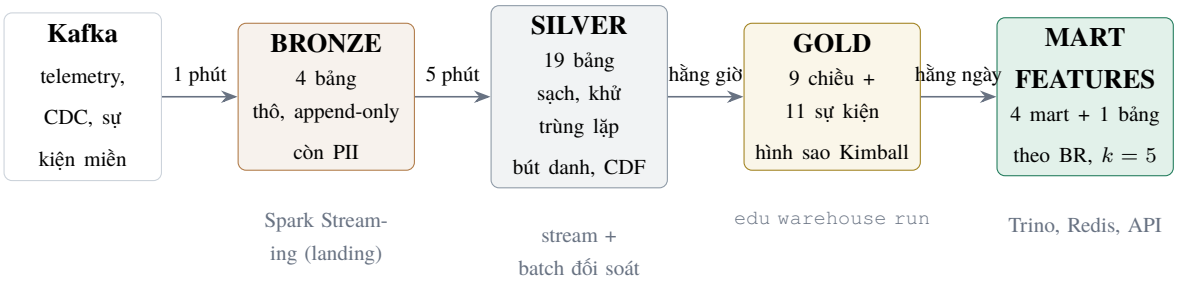
2. **Trino + Hive Metastore cho vai trò "Databricks"** (ADR 0010). Delta Lake chỉ là định dạng bảng. Vai trò danh mục và SQL warehouse của Databricks được lấp bằng Hive Metastore và Trino, chạy được cục bộ. Trên cloud có thể thay bằng Databricks SQL và Unity Catalog mà không đổi mô hình dữ liệu.
3. **Redis thay Cassandra** cho tầng phục vụ (ADR 0006). Đề bài cho phép chọn một trong hai. Redis đủ cho khối lượng hiện tại; adapter nằm sau cổng nên có thể thay khi dữ liệu vượt bộ nhớ.
4. **BR2 dùng luật làm trục, feature store làm ngữ cảnh.** Đề bài mô tả pipeline đặc trưng hai nhánh qua feature store. Nhóm đã hiện thực nhánh lô (`features.learner_longterm`) và nhánh phiên streaming, cả hai phục vụ qua Redis; gợi ý vẫn do luật xếp hạng, dùng đặc trưng để cổng/ưu tiên và giải thích (Mục 4.2). Mô hình học máy chưa nằm trong phạm vi.

CHƯƠNG 3

LAKEHOUSE VÀ PIPELINE DỮ LIỆU

Chương này trình bày phần cốt lõi phục vụ phân tích: lakehouse [16] Delta Lake tổ chức theo mô hình medallion [17], và kho dữ liệu chính là lớp gold của lakehouse. Nền tảng không có cơ sở dữ liệu phân tích thứ hai. Spark ghi các bảng Delta, Hive Metastore lập danh mục, Trino phục vụ truy vấn SQL, còn Redis phục vụ các tra cứu cấp mili-giây. Toàn bộ DDL, model biến đổi, cổng chất lượng và cấu hình nằm dưới dạng mã trong thư mục `warehouse/`.

3.1 Kiến trúc medallion



Hình 3.1. Các lớp của lakehouse và tần suất cập nhật

Bảng 3.1. Hợp đồng dữ liệu của từng lớp

Schema	Số bảng	Bên ghi	Hợp đồng
`bronze`	4	Stream landing	Mỗi bản ghi Kafka một dòng, giữ nguyên như nhận được. Chỉ append, còn PII, chỉ vai trò `pii_steward` đọc được. Giữ 30 ngày (telemetry) hoặc 90 ngày (CDC, sự kiện). Phát lại bronze dựng lại được mọi lớp sau.
`silver`	19	Stream silver + batch đối soát	Có kiểu, đã kiểm tra hợp lệ, khử trùng lặp, đúng schema hiện hành, đã khử định danh bằng HMAC; bỏ định danh trực tiếp. CDC áp dụng theo thứ tự LSN, xoá mềm. Bật Change Data Feed.

Schema	Số bảng	Bên ghi	Hợp đồng
gold	9 + 11	edu warehouse run	Lược đồ hình sao Kimball (ADR 0012): bảng chiều dùng chung, khoá thay thế tất định, bảng sự kiện có grain rõ ràng.
mart	4	--layer mart	Mỗi BR một bảng, theo nhu cầu người đọc; ngưỡng lấy từ policy của domain. mart.org_progress là bảng duy nhất quản trị viên B2B được đọc.
features	1	--layer features	Kho đặc trưng ngoại tuyến cho BR2, mỗi as_of_date một phân vùng để huấn luyện đúng thời điểm (point-in-time).
ops	2	Runner	pipeline_runs (mỗi lần build: cửa sổ, số dòng, phiên bản Delta) và dq_results (kết quả kiểm tra chất lượng).

3.2 Chuẩn hoá telemetry và tiến hoá schema

Telemetry đi qua use case NormalizeEvents, chạy giống hệt nhau trong job silver và trong job thời gian thực:

1. **Kiểm tra hợp lệ** theo hợp đồng trong schemas/: thiếu trường bắt buộc, động từ (verb) không thuộc từ vựng, hoặc payload hỏng thì bị đưa vào DLQ kèm lý do.
2. **Chuyển schema**: mỗi sự kiện mang schema_version. Chuỗi SchemaMigrator đưa phiên bản cũ (1.0) lên phiên bản hiện hành (1.1); phiên bản không biết bị từ chối.
3. **Khử định danh**: định danh người học được thay bằng bút danh; email, họ tên, IP, mã thiết bị bị loại bỏ.
4. **Thời gian**: tính độ trễ so với lúc tiếp nhận, gắn cờ late, và ghim các sự kiện có đồng hồ chạy trước.
5. **Khử trùng lặp**: MERGE theo event_id (chỉ chèn) loại các lần gửi lại từ thiết bị offline.

3.3 Khử định danh và phân quyền

Định danh thật chỉ tồn tại ở bronze (ADR 0007). Từ silver trở đi, người học được biểu diễn bằng bút danh dạng k<phiên bản khoá>_<24 ký tự hex>, tính bằng HMAC-SHA256 với khoá bí mật có phiên bản. Hàm SQL pseudonymize() dùng đúng thuật toán và khoá của HmacPseudonymizer trong context telemetry, nên dữ liệu telemetry, CDC và sự kiện miền vẫn

nối được với nhau. Cổng chất lượng `silver_no_raw_identifiers` chặn mọi định danh thô lọt qua bronze.

Trino áp dụng phân quyền chỉ đọc theo schema (Bảng 3.2). Quản trị viên B2B chỉ thấy đúng một bảng đã áp dụng k-anonymity.

Bảng 3.2. Phân quyền truy cập kho dữ liệu qua Trino

Người dùng	bronze	silver	gold	mart	features / ops
`pii_steward`	✓	✓	✓	✓	✓
`analyst`	–	✓	✓	✓	✓
`app` (API, dashboard)	–	–	✓	✓	–
`b2b_admin`	–	–	–	chỉ `org_progress`	–

3.4 Mô hình chiều Kimball ở lớp gold

Tám yêu cầu đọc các sự kiện chồng lấn nhau (sự kiện học tập, bài nộp, ghi danh, rủi ro) theo cùng các thực thể (người học, khoá học, nội dung, bài tập, tổ chức). Nếu không có bảng chiều dùng chung, mỗi mart sẽ tự suy ra các thực thể này và dần lệch nhau. Vì vậy lớp gold được mô hình hoá thành lược đồ hình sao theo phương pháp Kimball [18] (ADR 0012). Bảng 3.3 là ma trận bus cho biết mỗi bảng sự kiện dùng những chiều nào.

Bảng 3.3. Ma trận bus của lớp gold (D: vai trò ngày, ●: khoá chiều)

Bảng sự kiện (grain)	*date*	*learner*	*course*	*content*	*activity*	*lesson*	*assignment*	*test case*	*org*	**BR**
`fct_learning_event` (một sự kiện)	D	●	●	●	●					6,7,8
`fct_learner_daily_activity` (người học × khoá × ngày)	D	●	●							2,8
`fct_video_second_activity` (phiên bản × giây × ngày)	D			●						7
`fct_submission` (một lần nộp)	D	●	●				●			3,4
`fct_test_case_result` (lần nộp × test)	D	●					●	●		4
`fct_lesson_progress` (người học × bài)		●	●			●				6
`fct_enrollment` (một lần ghi danh)	D×5	●	●						●	5,6
`fct_risk_score_daily` (người học × khoá × ngày)	D	●	●							5
`fct_dropout_alert` (một cảnh báo)	D	●	●							5
`fct_notification` (một kết quả gửi)	D	●								1
`fct_content_sync` (một phiên bản nội dung)			●	●						8

3.4.1 Bảng chiều và khoá thay thế

- **SCD loại 2** cho `dim_learner` và `dim_content`. Với `dim_learner`, báo cáo B2B phải quy hoạt động về tổ chức tại thời điểm xảy ra. Với `dim_content`, mỗi phiên bản video có một dòng riêng; video quay lại có độ dài khác nên giây thứ n của hai phiên bản không cùng nghĩa (BR7). Các chiều còn lại dùng SCD loại 1.
- **Khoá thay thế tất định**: `xxhash64(khoá tự nhiên)` với SCD1 và `xxhash64(khoá tự nhiên, valid_from)` với SCD2. Dựng lại một bảng chiều sẽ tạo lại đúng các khoá cũ, nên bảng chiều và bảng sự kiện build độc lập, không cần dịch vụ sinh số tuần tự.

- **Thành viên suy luận** (inferred member) cho chiều đến muộn: telemetry của một người học mà dòng CDC chưa đến sẽ tạo một dòng `is_inferred = true`, mang đúng khoá mà dòng thật sẽ có. Khi dòng thật đến, nó ghi đè tại chỗ nên không bảng sự kiện nào phải đổi khoá. Đã kiểm chứng: người học `l-131` có sự kiện hai ngày trước dòng CDC vẫn giữ nguyên khoá qua quá trình chuyển đổi.
- Khoá `-1` biểu diễn "không áp dụng", ví dụ nội dung của một sự kiện đăng nhập.

Một số quy tắc nghiệp vụ dùng chung: bài học được coi là hoàn thành khi CDC báo `percent_done = 100` *hoặc* có sự kiện telemetry `completed`, tuỳ cái nào đến trước; vị trí video vượt quá độ dài phiên bản được kẹp về giây cuối; lần chạy sandbox mới nhất của một bài nộp thay thế các lần trước.

3.5 Data mart theo yêu cầu nghiệp vụ

Mỗi mart hiện thực lại một use case phân tích bằng SQL theo tập hợp. Các ngưỡng được truyền từ value object của domain (`policy_params`), ghi lại trên mỗi dòng và có test đơn vị đối chiếu với domain.

Bảng 3.4. Các data mart và kho đặc trưng

Bảng	BR / người đọc	Ngữ nghĩa	Policy
`mart.assessment_item_stats`	BR4, giảng viên	Tỷ lệ đạt theo (bài tập, test case) trong 28 ngày; bất thường khi có ≥ 30 lượt, tỷ lệ đạt $\leq 0{,}35$ và thấp hơn trung vị của bài tập ít nhất $0{,}30$	`AnomalyThreshold`
`mart.org_progress`	BR6, quản trị B2B	Theo tổ chức × khoá học: quy mô nhóm, tỷ lệ hoàn thành, mức thành thạo trung bình; **nhóm dưới k người chỉ công bố quy mô** [19]	`MinCohortPolicy(k=5)`
`mart.video_heatmap`	BR7, giảng viên	Lưới dày theo từng giây cho mỗi phiên bản video trong 28 ngày; số người học phân biệt chính xác; cường độ 0–1	–
`mart.engagement_trend`	BR8, giảng viên	4 cửa sổ tuần cho mỗi người học × khoá; "giảm" khi điểm giảm $\geq 30\%$ trong mỗi cửa sổ của hai tuần gần nhất	`EngagementWeights`, tỷ lệ 0,30

Bảng	BR / người đọc	Ngữ nghĩa	Policy
`features.learner_longterm`	BR2, gợi ý	`ComputeBatchFeatures` trong 30 ngày, theo `as_of_date`	registry đặc trưng v1

BR1, BR3 và BR5 được phục vụ thời gian thực (dịch vụ thông báo, dịch vụ chấm bài, runtime BR4/BR5 của ADR 0009). Kho dữ liệu giữ lịch sử phân tích bên cạnh đường phục vụ đó: `fct_risk_score_daily` và `fct_dropout_alert` có nhãn kết quả (`dropped_within_30d`) để sau này đánh giá độ chính xác của cảnh báo; `fct_notification` và `fct_submission` cho BR1 và BR3.

3.6 Thiết kế vật lý

- **Tính năng Delta**: ACID MERGE, `INSERT ... REPLACE WHERE` cho cửa sổ ngày, ghi đè phân vùng động, Change Data Feed ở silver, time travel (`FOR VERSION AS OF`) trong phạm vi VACUUM 7 ngày.
- **Phân vùng** chỉ theo cột ngày mà truy vấn lọc (ngày sự kiện, ngày hoạt động, ngày nộp, `as_of_date`); bảng nhỏ không phân vùng; bronze phân vùng theo `ingest_date`.
- **Gom cụm**: thuộc tính bảng `edu.zorder` được job bảo trì hằng ngày áp dụng bằng `OPTIMIZE ... ZORDER BY`. Cùng job này gộp các file nhỏ do writer streaming sinh ra.
- **Chính sách là thuộc tính bảng**: `edu.pii`, `edu.owner`, `edu.retention_days`, `edu.zorder` hiển thị cho mọi engine (`SHOW TBLPROPERTIES`, bảng `$properties` của Trino).
- **Tiến hoá schema tường minh**: tắt `autoMerge`; đổi DDL kèm câu lệnh migration.

3.7 Cổng kiểm tra chất lượng dữ liệu

Sau mỗi lớp, runner chạy các truy vấn kiểm tra trong `warehouse/quality` và ghi kết quả vào `ops.dq_results`. Cổng mức `error` dừng lần chạy.

Bảng 3.5. Các cổng kiểm tra chất lượng

Cổng	Mức	Bảo vệ điều gì
`silver_event_id_unique`	error	Khử trùng lặp các lần gửi lại từ mobile
`silver_no_raw_identifiers`	error	Ranh giới PII: chỉ bút danh sau bronze
`bronze_dlq_rate`	warn	Hơn 5% telemetry bị từ chối
`gold_dim_learner_scd2`	error	Mỗi người học đúng một phiên bản hiện hành, không chồng lấn hiệu lực
`gold_orphan_keys`	error	Mọi khoá trong bảng sự kiện tồn tại trong bảng chiều
`gold_unresolved_learners`	warn	Sự kiện bị quy về thành viên "không rõ"
`mart_org_progress_k_anonymity`	error	Không công bố tỷ lệ cho nhóm dưới k người
`mart_rates_in_range`	error	Các tỷ lệ nằm trong $[0, 1]$
`mart_org_progress_unresolved_dims`	warn	Dòng báo cáo thiếu khoá học hoặc kỹ năng
`mart_video_heatmap_unversioned`	warn	Ô heatmap không gắn phiên bản video

3.8 Điều phối, dữ liệu trễ và chạy lại

Các job streaming và dịch vụ chạy liên tục. Các job lô được Airflow điều phối (Bảng 3.6). Mọi lệnh lô đều idempotent theo `--run-date`, nên Airflow retry hay backfill đều an toàn, và `--start-date` dựng lại được bất kỳ khoảng thời gian nào.

Bảng 3.6. Các DAG Airflow

DAG	Lịch	Công việc
`medallion_daily`	hằng giờ	đối soát silver, build gold
`analytics_marts_daily`	02:00 hằng ngày	làm mới gold, mart BR4 / BR6 / BR7 cho ngày đã đóng
`engagement_weekly`	thứ Hai 02:00	`mart.engagement_trend`
`features_daily`	khi gold cập nhật	`features.learner_longterm`
`content_embedding`	6 giờ một lần	chia đoạn, nhúng học liệu
`lakehouse_maintenance`	04:00 hằng ngày	xoá theo thời hạn lưu, OPTIMIZE / Z-order, VACUUM

Bảng 3.7. Cơ chế xử lý dữ liệu trễ, trùng lặp và sai thứ tự

Tình huống	Lớp	Cơ chế
Kafka gửi lại, job chạy lại	bronze	Checkpoint Structured Streaming + sink Delta (đúng một lần)
Client gửi lại (mobile offline)	silver	MERGE theo `event_id`; cổng `silver_event_id_unique`
CDC sai thứ tự hoặc bị phát lại	silver	Lấy thay đổi mới nhất theo `lsn`; chỉ MERGE khi `lsn` mới hơn; phát lại không tạo commit Delta
Xoá ở OLTP	silver	Debezium chế độ `rewrite`: `is_deleted = true` (xoá mềm)
Sự kiện trễ (mobile đồng bộ sau vài ngày)	gold	Model theo cửa sổ khai báo `late_data`; cửa sổ tự lùi về ngày sự kiện cũ nhất mà silver nhận kể từ lần build trước (đã kiểm chứng: lần chạy 06–14/10 tự mở rộng về 04/10)
Sự kiện đến trước chiều	gold	Thành viên suy luận với khoá cuối cùng (ADR 0012)
Xoá theo thời hạn lưu ở bronze	silver	`skipChangeCommits` để thao tác xoá không làm dừng stream
Tiến hoá schema của client	silver	Chuỗi `SchemaMigrator` 1.0 → 1.1; không biết thì vào DLQ

Khi có sự cố, quy trình khôi phục dựa trên bronze là nhật ký phát lại: sửa SQL rồi chạy lại `edu warehouse run --start-date` cho lớp bị ảnh hưởng; khi cần trạng thái cũ thì dùng time travel của Delta với phiên bản lưu trong `ops.pipeline_runs`.

CHƯƠNG 4

HIỆN THỰC BR1–BR8 VÀ ỨNG DỤNG

Phần này đi qua tám yêu cầu nghiệp vụ, sau đó trình bày giao diện và cách vận hành demo/service.

Chương này đi qua tám yêu cầu nghiệp vụ. Với mỗi BR, báo cáo trình bày luồng dữ liệu, các quy tắc nghiệp vụ quan trọng và mức độ hoàn thiện. BR4–BR8 chạy trên hạ tầng thật (PostgreSQL, Debezium, Kafka, Spark, Delta, Trino, Redis, OpenSearch). BR1–BR3 có logic nghiệp vụ hoàn chỉnh và chạy được ở hai chế độ: runtime trong tiến trình cho demo (ADR 0008) và chế độ dịch vụ dùng Kafka/Redis/PostgreSQL, với API đóng vai trò BFF.

4.1 BR1 — Pipeline thông báo hướng sự kiện

Luồng. Hệ thống LMS phát các sự kiện `lms.assignment_created.v1`, `lms.deadline_approaching.v1`, `lms.course_updated.v1`; các context khác phát `grading.submission_graded.v1` và `risk.dropout_risk_detected.v1`. Dịch vụ thông báo tiêu thụ các topic này, đối chiếu với tập luật, xác định người nhận, rồi gửi qua các kênh in-app, push hoặc email. Kết quả (đã gửi hoặc bị chặn) được phát lên `notification.dispatched.v1` và `notification.suppressed.v1`, rồi đi vào lakehouse thành `gold.fct_notification`.

Rule engine. Luật được khai báo trong YAML và nạp lại nóng được (Hình 4.1). Điều kiện viết bằng một ngôn ngữ biểu thức rất nhỏ trên payload của sự kiện, không cho chạy mã tuỳ ý.

```yaml
rules:
  - rule_id: deadline-24h
    event_type: lms.deadline_approaching.v1
    condition: "hours_to_deadline <= 24"
    channels: [push, email]
    priority: high
    template_id: deadline_soon
  - rule_id: dropout-risk-advisor
    event_type: risk.dropout_risk_detected.v1
    channels: [email]      # recipient = the learner's advisor
    priority: high
```

Hình 4.1. Trích tập luật thông báo (`infra/notification/rules.yaml`)

Chống làm phiền. Mỗi thông báo có một khoá khử trùng lặp, lưu trong Redis với TTL 24 giờ, nên cùng một sự kiện không bao giờ gửi hai lần. Chính sách giới hạn tần suất mặc định là tối đa

5 thông báo mỗi giờ cho mỗi người học. Thông báo ưu tiên cao (ví dụ còn dưới 24 giờ đến hạn) được bỏ qua giới hạn này. Thông báo bị chặn được ghi lại kèm lý do để giao diện hiển thị.

Mức hoàn thiện. Rule engine, chính sách và use case đã hoàn chỉnh và có test. Dịch vụ tiêu thụ Kafka ghi thông báo vào Redis (item, inbox, dedup, rate limit) và API của giao diện đọc lại mô hình này khi chạy ở chế độ dịch vụ; một cầu SSE chuyển sự kiện `notification.dispatched.v1` tới trình duyệt. Kênh email dùng SMTP và kênh push dùng webhook/provider sau cùng một cổng `NotificationSender`; khi chưa cấu hình provider, kênh tương ứng chỉ ghi log. Phần còn lại là ranh giới sandbox thật và thông tin xác thực provider production.

4.2 BR2 — Gợi ý khoá học và bước học tiếp theo

Đề bài mô tả một pipeline đặc trưng hai nhánh: đặc trưng dài hạn tính theo lô bằng Spark, kết hợp đặc trưng phiên tính theo luồng, qua feature store. Nhóm đã hiện thực cả hai nhánh. Nhánh lô ghi bảng `features.learner_longterm` gồm tỷ lệ hoàn thành 30 ngày, tỷ lệ đạt trung bình, số ngày hoạt động, nền tảng ưa dùng và mức thành thạo theo kỹ năng, mỗi ngày một phân vùng để tránh rò rỉ dữ liệu tương lai; job `features-online` đẩy phân vùng mới nhất lên Redis. Nhánh phiên chạy bằng Spark Structured Streaming trên `telemetry.learning_events.v1`, chia phiên theo khoảng nghỉ 30 phút, tính số phút hoạt động, số sự kiện, các động từ gần nhất, khoá hiện tại và điểm lúng túng; trạng thái phiên nhỏ được giữ trong Redis và mỗi sự kiện được đánh dấu bằng `SET NX` để replay không đếm hai lần. Job cũng phát `features.session.v1` cho kho dữ liệu.

Bộ gợi ý vẫn là **gợi ý dựa trên luật**, tính tại thời điểm truy vấn từ danh mục khoá học và các khoá học viên đã hoàn thành. Luật xếp các khoá chưa ghi danh theo *tầng*, sau đó theo mức liên quan, độ khó và độ ưa thích (Bảng 4.1). Hai nhánh đặc trưng được dùng làm ngữ cảnh có kiểm soát: nếu tỷ lệ đạt gần đây thấp, bước nhảy độ khó bị hạ sau bước mở rộng cùng mức; phiên đang lúng túng ưu tiên bù kiến thức; phiên đang hoạt động cùng lộ trình được ưu tiên khi hoà. Mỗi gợi ý luôn kèm một câu giải thích để người học hiểu vì sao. Mô hình học máy vẫn nằm ngoài phạm vi.

Bảng 4.1. Các tầng của luật gợi ý (BR2)

Tầng	Trường hợp	Ví dụ lời giải thích	Điểm
0	Bước tiếp theo trong lộ trình	"Tiếp tục lộ trình data engineering: mức khó tiếp theo" hoặc "Bước liên quan trong lộ trình"	1,00
1	Mở rộng cùng mức	"Mở rộng ở mức của bạn: một khoá analytics mức dễ"	0,75
2	Bù khoá bị bỏ qua	"Bổ sung khoá mức dễ bạn đã bỏ qua"	0,50
3	Quá sức	"Quá sức: hoàn thành khoá mức trung bình trước" (khoá bị khoá)	0,00

4.3 BR3 — Tiếp nhận bài nộp và phản hồi chi tiết

Luồng tách rời. Bài nộp được đưa vào hàng đợi `grading.submission_received.v1` thay vì chấm ngay trong request, nên lượng nộp tăng đột biến trước hạn không làm nghẽn hệ thống. Worker sandbox chạy bài, ghi nhật ký thô lên `grading.sandbox_result.raw.v1`. Dịch vụ chấm bài chuẩn hoá kết quả và phát `grading.submission_graded.v1`, sự kiện này kích hoạt cả thông báo BR1.

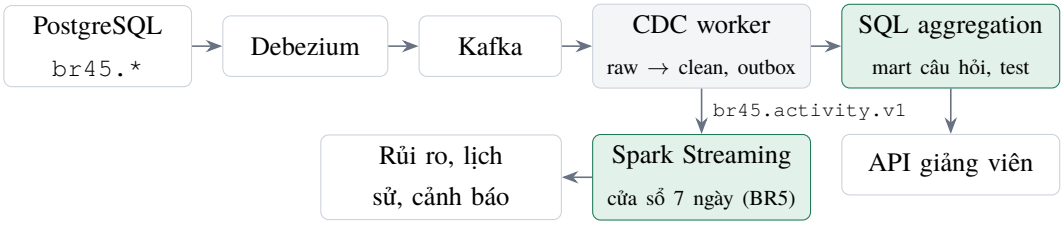
Sở hữu dữ liệu. Kết quả bền của BR3 nằm trong schema `grading` (`grading.submissions`, `grading.grades`), không ghi trực tiếp vào bảng LMS `public.*`. Một projector tùy chọn nghe `grading.submission_received.v1` và `grading.submission_graded.v1` để đồng bộ sang `public.submissions`/`public.grades` cho CDC và kho dữ liệu. Mặc định projector tắt để BR4–BR8 giữ nguyên baseline; khi bật, đây là tích hợp có chủ đích và phải tính lại số liệu báo cáo.

Sandbox. Bài Python chạy trong tiến trình con có giới hạn thời gian 3 giây. Bài SQL chạy trên SQLite trong bộ nhớ ở chế độ chỉ đọc, có hàm ngắt khi quá hạn. Lỗi hạ tầng được tách khỏi lỗi trong mã của người học: lỗi và timeout của mã người học tính là trượt, còn lỗi hạ tầng không tính vào tỷ lệ đạt.

Phản hồi có cấu trúc. `FeedbackPolicy` là một hàm thuần, không có I/O, tạo phản hồi gồm: câu tóm tắt, danh sách test trượt kèm giá trị mong đợi và thực tế (hoặc diff), 8 dòng cuối của stderr, và gợi ý theo ngôn ngữ dựa trên một bảng mẫu lỗi thường gặp. Kết quả còn gồm thời gian chạy, bộ nhớ và mã thoát. Mục 4.9 cho thấy cách phản hồi này hiển thị trong Code lab.

4.4 BR4 — Phát hiện câu hỏi và test case bất thường

Luồng CDC thật (ADR 0009). Dữ liệu vận hành nằm trong schema `br45` của PostgreSQL: khoá học, bài tập, câu hỏi và test case có phiên bản, bài nộp và kết quả chấm. Một connector Debezium riêng bắt WAL vào các topic `br45.cdc.br45.<bảng>`. Worker CDC lưu phong bì gốc vào `analytics.raw_changes`, kiểm tra hợp đồng (sai thì vào `quarantine`), khử trùng lặp theo định danh ổn định gồm bảng, LSN, giao dịch, thao tác và ảnh dòng, rồi ghi `clean_rows`. Thay đổi có LSN cũ không thể làm sống lại bản ghi mới hơn.



Hình 4.2. Đường CDC thật của BR4 và BR5

Ngữ nghĩa chỉ số. Mọi khoảng quan sát là nửa mở $[t_0, t_1)$. Một lần chấm test hợp lệ khi bài nộp đã được chấm, kết quả hợp lệ và trạng thái thuộc {passed, failed, error, timeout}. Một lần làm câu hỏi hợp lệ khi mọi test bắt buộc của phiên bản câu hỏi *tại thời điểm nộp* đều có kết quả hợp lệ, và chỉ đạt khi mọi test bắt buộc đều đạt. Phiên bản test mới hơn không âm thầm diễn giải lại các bài nộp cũ.

$$\text{pass_rate} = \frac{\text{số lần chấm hợp lệ đạt}}{\text{tổng số lần chấm hợp lệ}}$$

Mẫu số bằng 0 cho kết quả `null` (hiển thị là chưa có số liệu). Hệ thống cũng tính tỷ lệ đạt ở lần làm đầu tiên và lần làm cuối cùng của mỗi học viên.

Bảng 4.2. Quy tắc phát hiện bất thường của BR4 (cấu hình được)

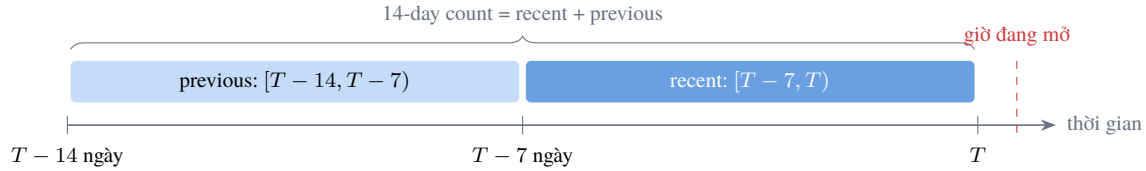
Quy tắc	Mặc định	Trạng thái sinh ra
Số học viên hợp lệ phân biệt tối thiểu	20	`INSUFFICIENT_DATA` nếu thiếu
Tỷ lệ đạt thấp của câu hỏi	dưới 30%	`LOW_PASS_RATE`
Tỷ lệ đạt thấp của test case	dưới 20%	`LOW_PASS_RATE`
Thấp hơn trung vị các test bắt buộc khác	$\geq$ 50 điểm %	`SUSPICIOUS_TEST_CASE`
Giảm so với cửa sổ liền trước cùng độ dài	$\geq$ 30 điểm %	`NEEDS_REVIEW`

Kết quả lưu trong ba mart (`mart_question_pass_statistics`, `mart_test_case_pass_statistics`, `mart_question_anomalies`), kèm định danh, phiên bản, cửa sổ, số đếm, lý do

và ngưỡng. Mỗi thay đổi CDC làm mới bản tổng hợp 30 ngày mặc định bằng cách thay giá trị tuyệt đối theo khoá ổn định, thay vì cộng dồn bộ đếm. Vì vậy việc sửa điểm hay xoá bản ghi được phản ánh đúng mà không cần chạy lại seed. Một cờ bất thường chỉ là yêu cầu giảng viên xem xét, không phải kết luận rằng đề bị lỗi.

4.5 BR5 — Cảnh báo sớm nguy cơ bỏ học

Cửa sổ trượt theo thời gian sự kiện. Spark Structured Streaming đọc `br45.activity.v1`, dùng watermark 21 ngày và `dropDuplicatesWithinWatermark(event_id)`, rồi tổng hợp bằng `window(event_time, '7 days', slide)` với bước trượt mặc định 1 giờ. Đây là phép tổng hợp cửa sổ có trạng thái thật sự, không phải một truy vấn SQL chạy theo lịch. Với mốc đánh giá T căn theo giờ UTC, các đặc trưng được tính như Hình 4.3.



Hình 4.3. Các cửa sổ đặc trưng tại mốc đánh giá T (BR5)

Cập nhật một hoạt động sẽ rút phần đóng góp cũ khỏi các cửa sổ rồi cộng phần mới; xoá thì rút phần đóng góp. Sự kiện "đồng hồ" có trọng số 0 tạo các cửa sổ rỗng, để tình trạng không hoạt động vẫn được ghi nhận khi học viên không làm gì. Mỗi micro-batch upsert tổng tuyệt đối của cửa sổ trong một giao dịch. Việc chấm điểm rủi ro chờ đến khi offset Spark đã commit bao phủ hết outbox, để tránh cảnh báo sớm trên đặc trưng chưa đầy đủ.

Điều kiện xét. Học viên phải ghi danh đang hiệu lực ít nhất 7 ngày. Bài tập bắt buộc chỉ tính khi hạn nộp nằm trong 14 ngày gần nhất và sau ngày ghi danh; hạn trong tương lai không bị phạt. Riêng việc thiếu telemetry không bị coi là không hoạt động: tín hiệu này đòi hỏi đã từng quan sát thấy hoạt động thật.

Bảng 4.3. Các tín hiệu rủi ro và điểm (BR5)

Tín hiệu	Điều kiện	Điểm
Không hoạt động	Ít nhất 7 ngày kể từ hoạt động có ý nghĩa gần nhất	35
Trễ hạn	Ít nhất một bài tập bắt buộc đã đến hạn mà chưa nộp	30
Tỷ lệ nộp thấp	Tỷ lệ nộp các bài đã đến hạn dưới 50%	20
Hoạt động giảm	Hoạt động giảm ít nhất 50%, với ít nhất 3 quan sát ở cửa sổ trước	15

Điểm tối đa là 100. Các mức: LOW 0–29, MEDIUM 30–59, HIGH 60–79, CRITICAL 80–100. Mọi kết quả kèm đặc trưng, lời giải thích, điều kiện xét, thời điểm đánh giá và chính sách. **Điểm không phải xác suất bỏ học**; đây là công cụ giúp cố vấn ưu tiên.

Vòng đời cảnh báo. Mỗi cặp học viên × khoá học chỉ có một cảnh báo mở. Vượt 60 điểm sinh cảnh báo mới; thời gian chờ mặc định là 72 giờ đánh giá. Cảnh báo đang mở được cập nhật mức độ và lý do hiện tại. Cố vấn có thể xác nhận (acknowledge) hoặc giải quyết (resolve); hệ thống lưu thời điểm và người thực hiện. Khi điểm giảm dưới ngưỡng, cảnh báo tự động được giải quyết. Lịch sử mọi lần đánh giá và mọi lần sửa đều được giữ lại để kiểm toán.

4.6 BR6 — Tiến độ kỹ năng theo tổ chức

BR6 là ví dụ đầy đủ nhất của pipeline medallion. Telemetry và CDC đi qua silver (chuyển schema, khử định danh), lên gold (`fct_enrollment`, `fct_lesson_progress`, `fct_submission`), rồi tổng hợp thành `mart.org_progress` theo tổ chức × khoá học. Tỷ lệ hoàn thành là tỷ lệ bài học đã xong trung bình của khoá. Mức thành thạo là tỷ lệ đạt test trung bình của các bài nộp đã chấm; học viên chưa có bài chấm được tính là 0.

k-anonymity được bảo vệ ba lớp. (1) Mart chỉ công bố quy mô cho nhóm dưới $k = 5$ người, các tỷ lệ để trống. (2) Cổng chất lượng mức error `mart_org_progress_k_anonymity` làm hỏng lần build nếu vi phạm. (3) Bộ đọc của API xoá các tỷ lệ của dòng bị ẩn, kể cả khi mart lỗi vẫn mang giá trị. Token quản trị viên chỉ được phép đọc các tổ chức trong phạm vi của mình; truy cập ngoài phạm vi trả về 403.

4.7 BR7 — Heatmap tương tác video

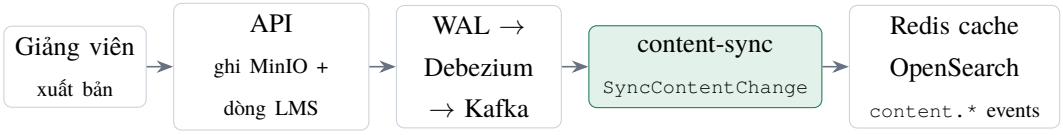
Theo phiên bản. Sự kiện được quy về phiên bản nội dung có hiệu lực tại thời điểm xảy ra (`gold.dim_content` là SCD2). Vì vậy video quay lại có heatmap riêng cho mỗi phiên bản, và API mặc định chọn phiên bản mới nhất. Bảng `gold.fct_video_second_activity` có grain phiên bản × giây × ngày, đếm play, pause, seek và số người học phân biệt. `mart.video_heatmap` tạo lưới dày cho từng giây trong 28 ngày, có số người học phân biệt chính xác và cường độ chuẩn hoá 0–1. Vị trí vượt quá độ dài video được kẹp về giây cuối.

Lớp phủ hôm nay (ADR 0014). Ứng dụng `telemetry-live` chạy ba truy vấn Spark có checkpoint. Truy vấn thứ nhất chuẩn hoá telemetry thô bằng đúng `normalize_rows` của silver. Hai truy vấn còn lại tổng hợp theo giây video (BR7) và điểm tương tác theo ngày (BR8), với watermark 5 phút và khử trùng lặp theo `event_id`. Kết quả ghi vào Redis dưới dạng giá trị tuyệt đối, nên phát lại sẽ ghi đè chứ không cộng thêm. API chỉ ghép lớp phủ cho những ngày sau ảnh chụp của mart, và chỉ cho phiên bản mới nhất. Nếu Redis lỗi, API trả về dữ liệu mart.

4.8 BR8 — Đồng bộ nội dung và xu hướng tương tác

4.8.1 Đồng bộ cache và chỉ mục tìm kiếm

Một phiên bản nội dung mới là một lần cập nhật dòng `public.course_content`, trỏ tới một đối tượng bất biến mới trong MinIO (`courses/<khoá>/<nội dung>/v<n>.<ext>`). API của giảng viên chỉ ghi đối tượng và dòng dữ liệu, dưới khoá dòng để tránh hai người xuất bản cùng lúc, chứ không tự đồng bộ. Debezium bắt thay đổi, và dịch vụ `content-sync` chạy use case `SyncContentChange` (Hình 4.4).



Các bước trong content-sync:

1. Lấy đối tượng 2. Vô hiệu hoá cache Redis 3. Chia đoạn + nhúng vector 4. Dọn phiên bản khác
5. Làm mới mục tìm kiếm 6. Làm ấm cache 7. Phát sự kiện

Hình 4.4. Đồng bộ nội dung BR8 dựa trên CDC

Idempotent theo trạng thái chỉ mục, không theo thứ tự đến. `sync_policy` chỉ áp dụng phiên bản mới hơn phiên bản đã chỉ mục; bằng nhau là trùng lặp, cũ hơn là lỗi thời. Do đó phát lại CDC, khởi động lại hay backfill đều an toàn. Tìm kiếm chỉ trả các đoạn thuộc phiên bản hiện hành của mỗi nội dung, nên việc dọn dẹp chậm không bao giờ làm lộ văn bản cũ.

Tìm kiếm lai. OpenSearch lưu các đoạn văn bản với chỉ mục BM25 và vector `knn_vector` (HNSW [20]). Kết quả hai phía được hợp nhất bằng reciprocal rank fusion [21] phía client. Vector hiện được sinh bởi một bộ nhúng băm tất định (không cần tải mô hình, không cần GPU), nằm sau cổng `Embedder`; thay bằng mô hình sentence-embedding chỉ cần một adapter mới và một lần reindex. Job Spark `edu batch content-reindex` phân phối *cùng* use case theo từng nhóm nội dung để backfill và ghi `silver.content_chunks`.

4.8.2 Xu hướng tương tác

Điểm tương tác của mỗi học viên trong một cửa sổ là tổng có trọng số của các hoạt động (Bảng 4.4). Mart `mart.engagement_trend` tính 4 cửa sổ tuần (thứ Hai đến Chủ nhật) cho mỗi học viên × khoá học.

Bảng 4.4. Trọng số hoạt động trong điểm tương tác (`EngagementWeights`)

Nhóm	Động từ (verb)	Trọng số
Đăng nhập	`logged_in`	1,0
Xem bài	`viewed`, `played`, `completed`	1,0
Làm bài	`attempted`, `submitted`	2,0
Thảo luận	`posted`	1,5

Gọi a, b, c là điểm của ba cửa sổ gần nhất theo thứ tự thời gian, và $r = 0{,}30$ là tỷ lệ giảm. Học viên được đánh dấu *đang giảm* khi

$$a > 0 \ \wedge \ b \le a\,(1 - r) \ \wedge \ c \le b\,(1 - r),$$

tức là điểm giảm ít nhất 30% trong *mỗi* cửa sổ của hai tuần liên tiếp. Lớp phủ "hôm nay" dùng đúng các trọng số này, nên giá trị trong ngày bằng giá trị mà kho dữ liệu sẽ tính vào ngày hôm sau.

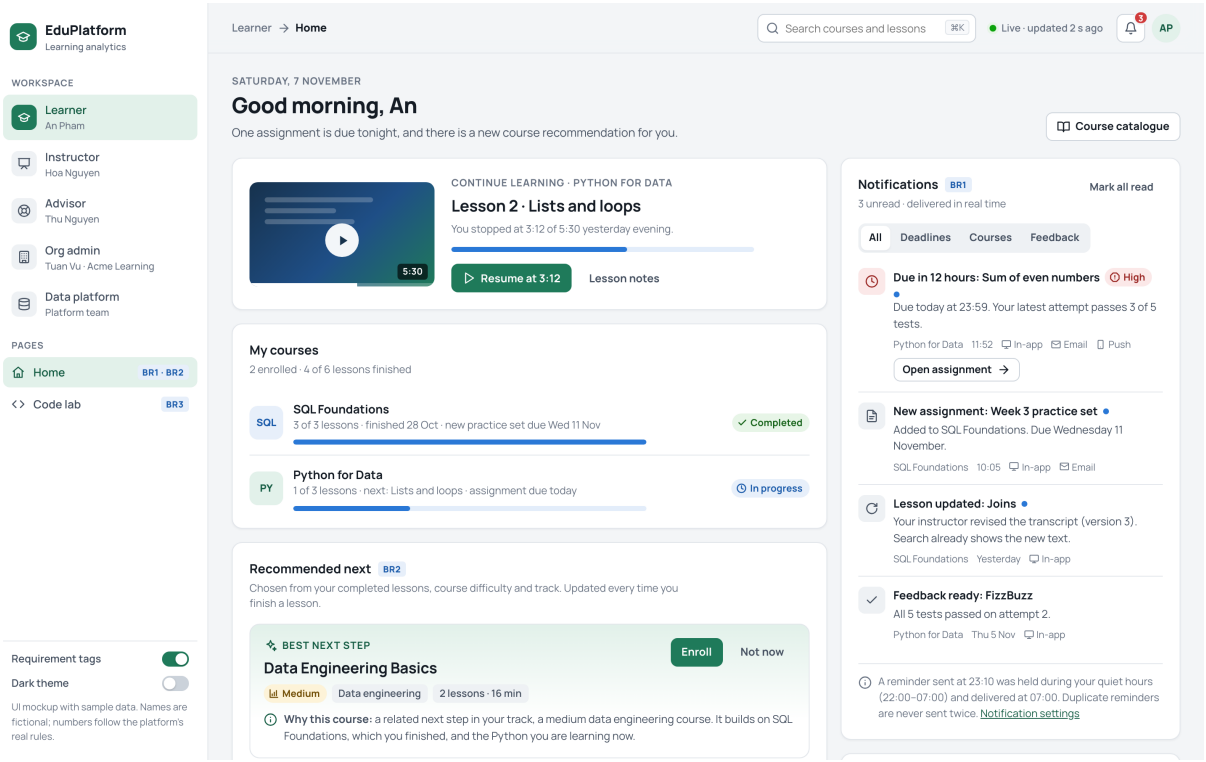
4.9 Giao diện người dùng

Pipeline dữ liệu chỉ có giá trị khi kết quả của nó đến được tay người dùng. Phần này trình bày ứng dụng web phục vụ năm nhóm người dùng: người học, giảng viên, cố vấn học tập, quản trị viên tổ chức và đội vận hành nền tảng dữ liệu.

4.9.1 Quy trình thiết kế

Phiên bản đầu của ứng dụng web là một bảng điều khiển demo hướng tới nhà phát triển: các nút giả lập sự kiện, mã định danh thô và ô nhập token nằm lẫn với kết quả. Nhóm thiết kế lại theo hai bước:

1. **Mockup HTML tương tác** (`docs/ui-mockup/edu-platform-ui-mockup.html`): một file độc lập, dùng dữ liệu mẫu nhưng tuân theo đúng các quy tắc nghiệp vụ thật (điểm rủi ro 35/30/20/15, ngưỡng 30/60/80, $k = 5$, tỷ lệ giảm 30%). Mockup giúp thống nhất bố cục và nội dung trước khi viết mã.
2. **Hiện thực bằng React 18 + TypeScript** trong `apps/web`, giữ nguyên Vite, không thêm thư viện giao diện hay biểu đồ. Mọi màn hình đọc API thật. Những phần của mockup không có API tương ứng (độ tươi của báo cáo, kết quả kiểm tra chất lượng, ghi chú can thiệp) được bỏ ra thay vì dùng dữ liệu giả.



[TODO: thay bằng screenshot mới (hoặc giữ mockup có chủ đích)]

Hình 4.5. Mockup: trang chủ người học với thông báo (BR1) và gợi ý khoá học kèm lý do (BR2)

4.9.2 Hệ thống thiết kế

- **Token thiết kế** cho màu, khoảng cách, bo góc và bóng đổ, có bộ giá trị riêng cho chế độ sáng và tối. Người dùng chuyển chế độ bằng một công tắc; mặc định theo hệ điều hành.
- **Bảng màu biểu đồ đã kiểm định**: màu phân loại theo thứ tự cố định và một dải màu xanh đơn sắc cho độ lớn (heatmap), đã kiểm tra khả năng phân biệt với người mù màu. Màu trạng thái (tốt, cảnh báo, nghiêm trọng, nguy hiểm) chỉ dùng cho trạng thái và luôn đi kèm biểu tượng và nhãn chữ, không bao giờ chỉ dựa vào màu.
- **Khả năng tiếp cận**: mọi biểu đồ có tooltip khi rê chuột và một bảng dữ liệu "Xem dạng bảng" thay thế; các nút có nhãn cho trình đọc màn hình; bố cục co giãn tới màn hình điện thoại mà không phải cuộn ngang.
- **Nhãn yêu cầu**: mỗi trang và mỗi khối được gắn nhãn BR tương ứng (có thể tắt). Nhãn này giúp truy vết từ giao diện về yêu cầu khi trình bày.

4.9.3 Kiến trúc frontend

Ứng dụng tổ chức theo *không gian làm việc* (workspace), mỗi nhóm người dùng một workspace, chọn ở thanh bên (Bảng 4.5). Thanh trên cùng có chỉ báo sự kiện trực tiếp (Server-Sent Events), thông báo của người học và trạng thái đăng nhập.

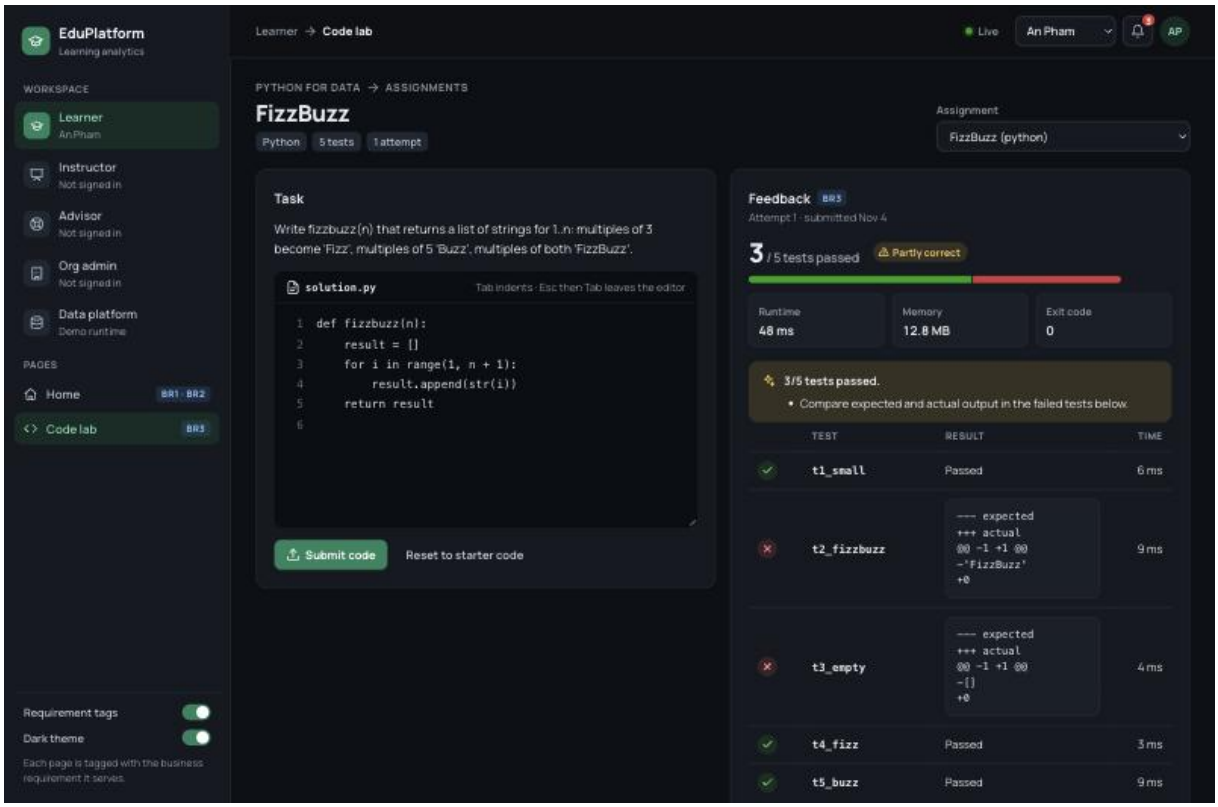
Bảng 4.5. Workspace, đường dẫn và trang

Workspace	Đường dẫn	Nội dung	BR
Learner	`/learner`	Khoá học, gợi ý, phản hồi gần đây, thông báo	1, 2
Learner	`/learner/code-lab`	Trình soạn mã, phản hồi từng test, các lần nộp	3
Instructor	`/instructor/assessments`	Câu hỏi và chi tiết test case	4
Instructor	`/instructor/videos`	Heatmap theo phiên bản, lớp phủ hôm nay, thời điểm cần xem lại	7
Instructor	`/instructor/engagement`	Trung bình khoá học, học viên đang giảm	8
Instructor	`/instructor/content`	Xuất bản phiên bản, theo dõi đồng bộ, xem trước tìm kiếm	8
Advisor	`/advisor`	Hàng đợi rủi ro, phân rã điểm, lịch sử, cảnh báo	5
Org admin	`/admin`	Tiến độ tổ chức có ẩn nhóm nhỏ, xuất CSV	6
Data platform	`/pipeline`	Bộ đếm luồng, thông lượng, topic, DLQ, silver	–

Các workspace Instructor, Advisor và Org admin được bảo vệ bằng một thẻ đăng nhập nhận token của vai trò. Token được giữ trong phiên của tab trình duyệt và dùng chung cho mọi trang của workspace. Máy chủ tự ánh xạ token sang danh tính, vai trò và phạm vi khoá học hoặc tổ chức; giao diện không tự khai báo quyền. Các điều khiển chỉ dành cho demo (giả lập sự kiện LMS, bộ sinh telemetry) được tách vào các khối “Demo controls” viền nét đứt. Biểu đồ được vẽ bằng SVG thuần với chiều rộng thật của phần tử, nên chữ không bị co giãn.

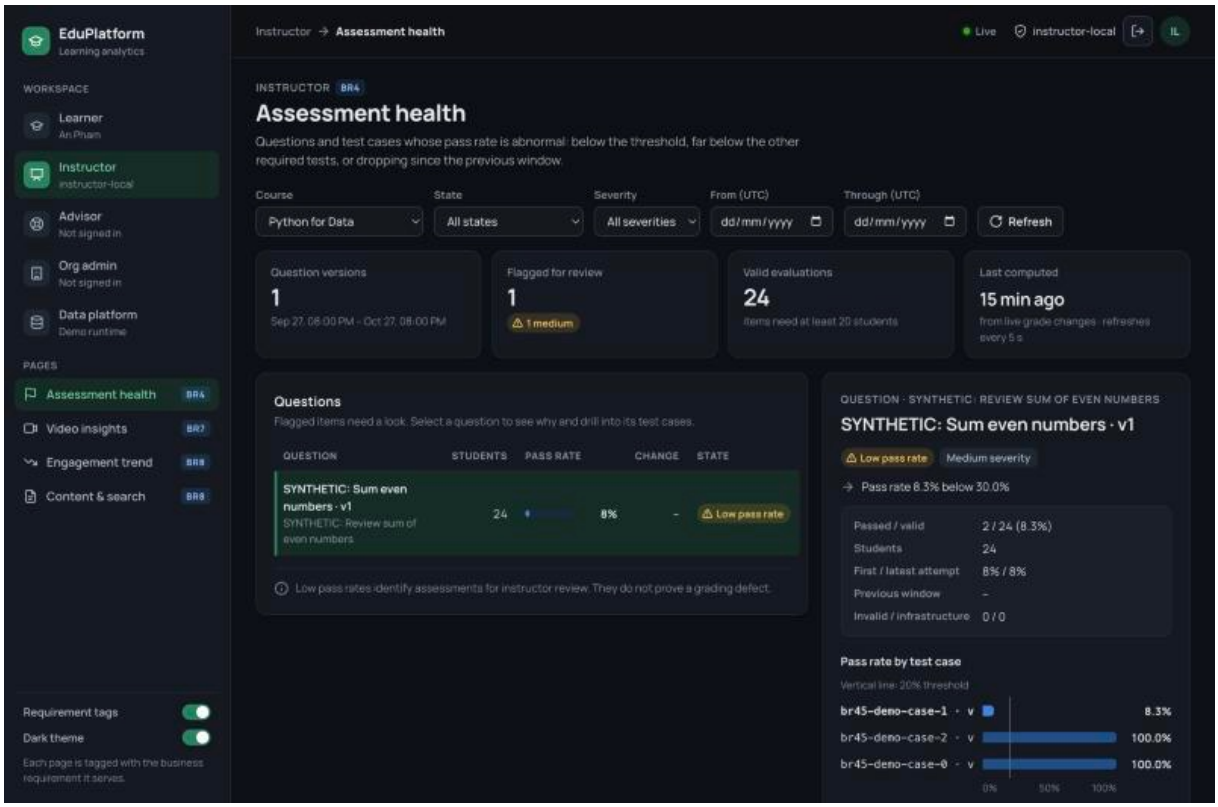
4.9.4 Các màn hình chính

Các ảnh trong mục này được chụp từ ứng dụng thật chạy trên dữ liệu của hệ thống (chế độ tối), trừ khi ghi là mockup.



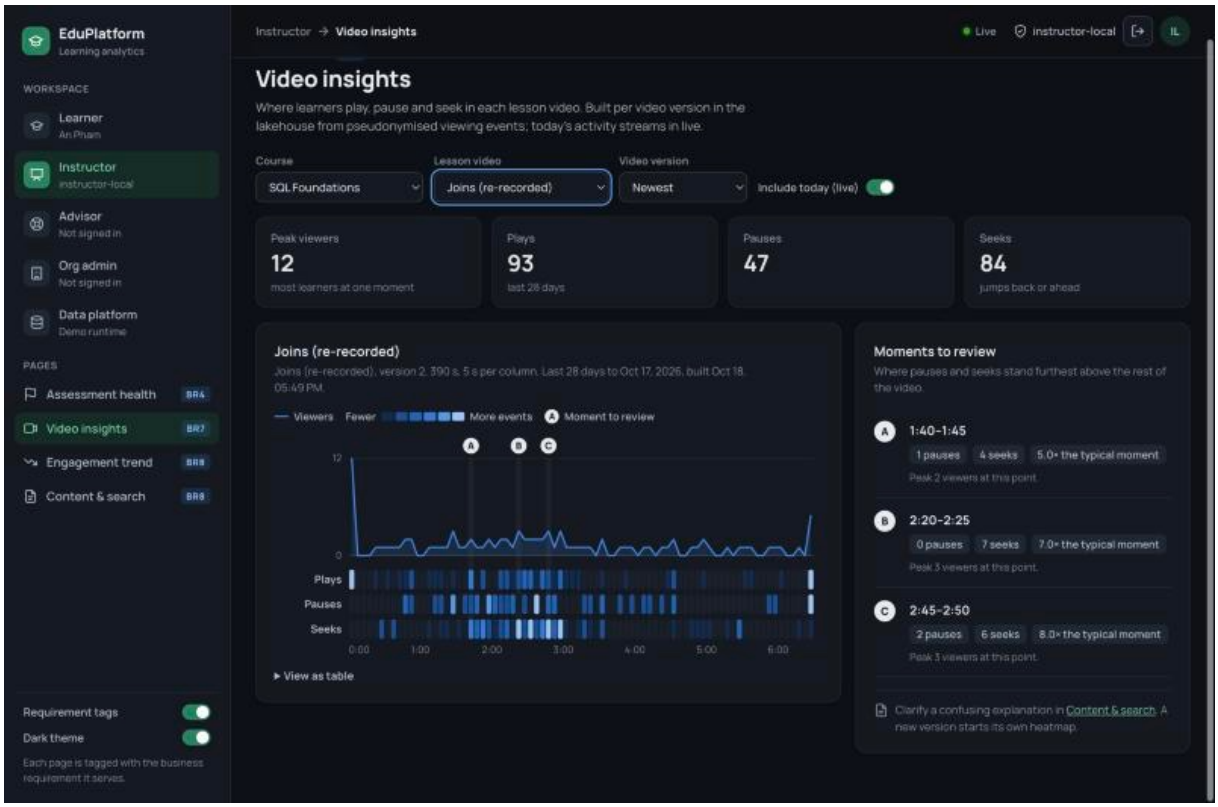
[TODO: chụp lại ảnh mới]

Hình 4.6. Code lab (BR3): bài nộp FizzBuzz được chấm trong sandbox, phản hồi từng test kèm diff



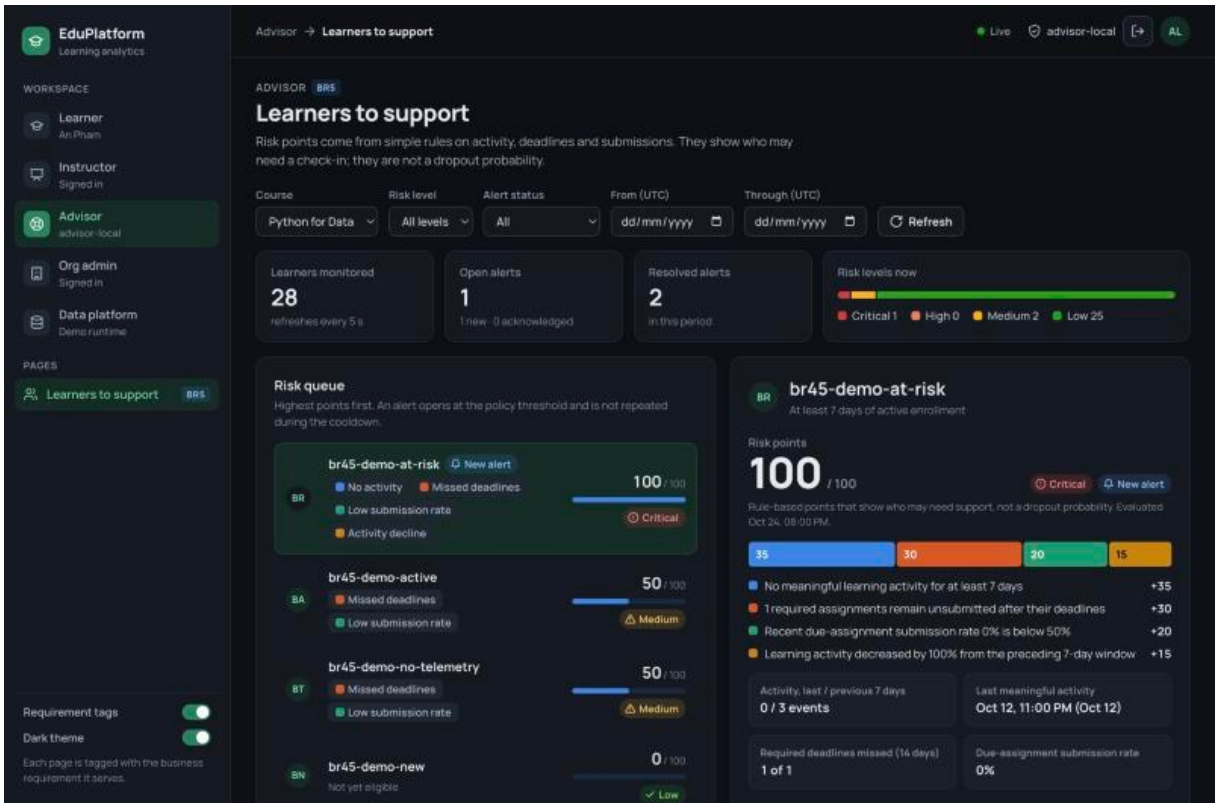
[TODO: chụp lại ảnh mới]

Hình 4.7. Assessment health (BR4): câu hỏi bị gắn cờ và tỷ lệ đạt theo từng test case so với ngưỡng 20%



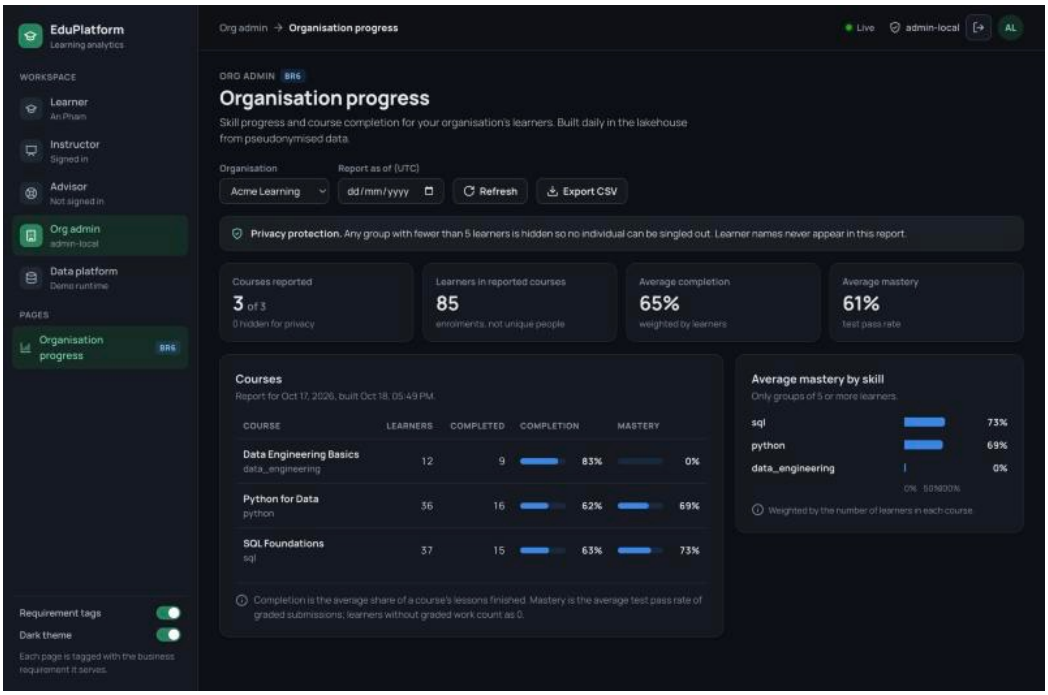
[TODO: chụp lại ảnh mới]

Hình 4.8. Video insights (BR7): heatmap play/pause/seek của phiên bản 2 video “Joins” (390 giây), kèm các thời điểm cần xem lại



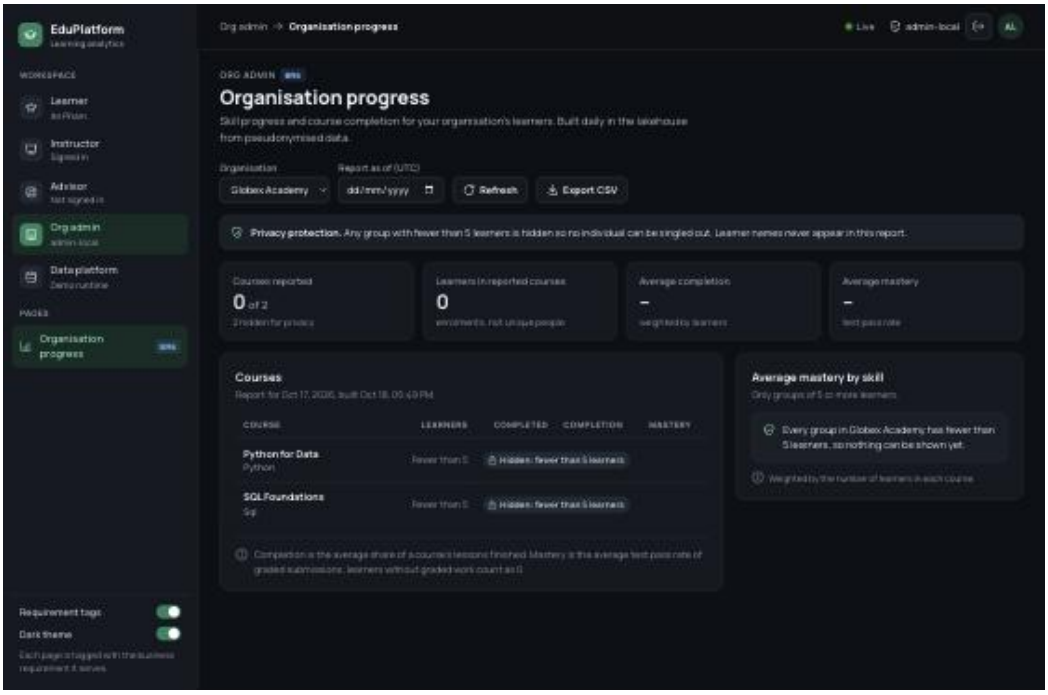
[TODO: chụp lại ảnh mới]

Hình 4.9. Learners to support (BR5): hàng đợi rủi ro và phân rã 100 điểm thành 35 + 30 + 20 + 15



[TODO: chụp lại ảnh mới]

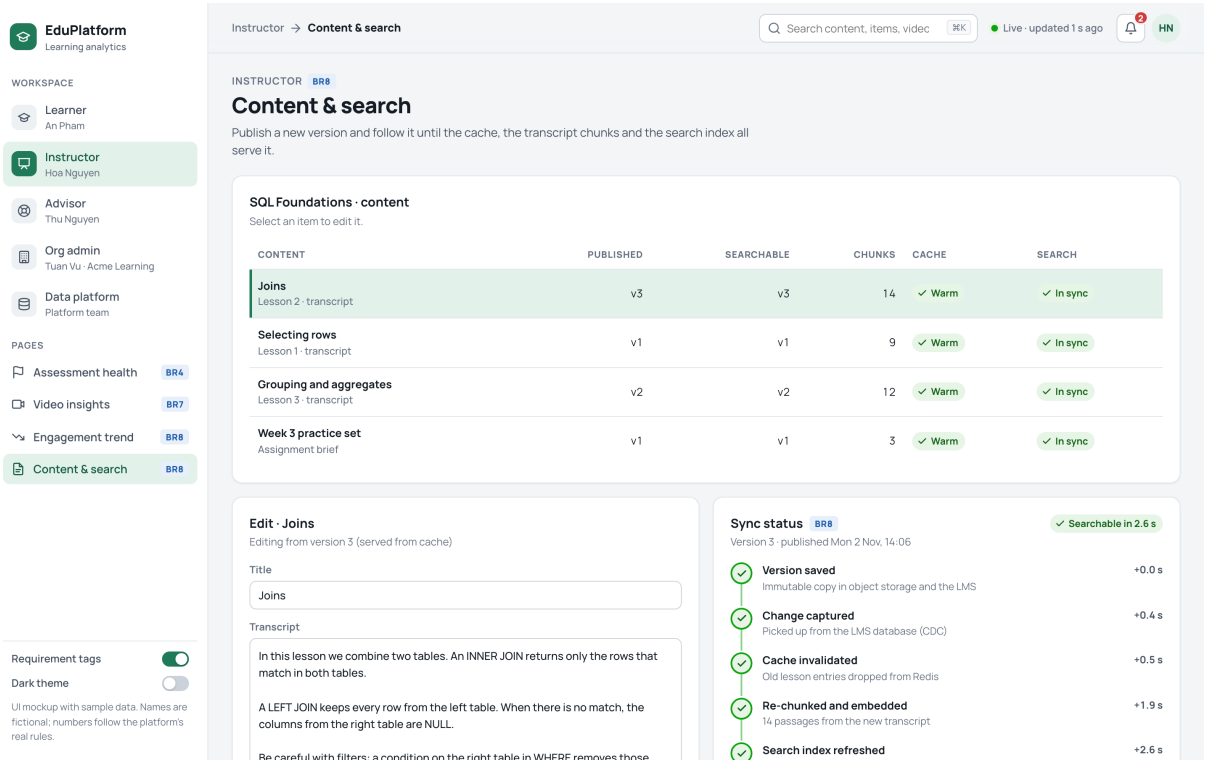
(a) Acme Learning: mọi nhóm đủ 5 người nên được báo cáo



[TODO: chụp lại ảnh mới]

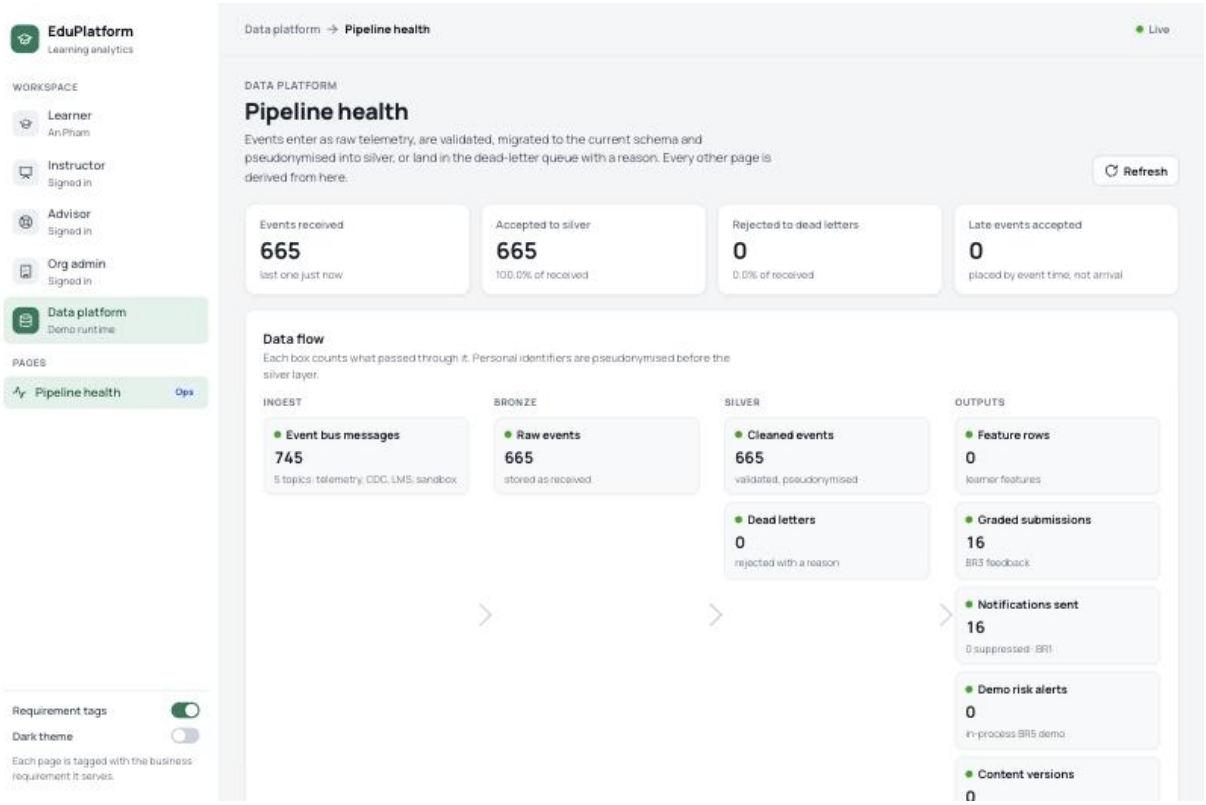
(b) Globex Academy: mọi nhóm dưới 5 người nên bị ẩn, không lộ tỷ lệ hay quy mô chính xác

Hình 4.10. Organisation progress (BR6): k-anonymity với $k = 5$



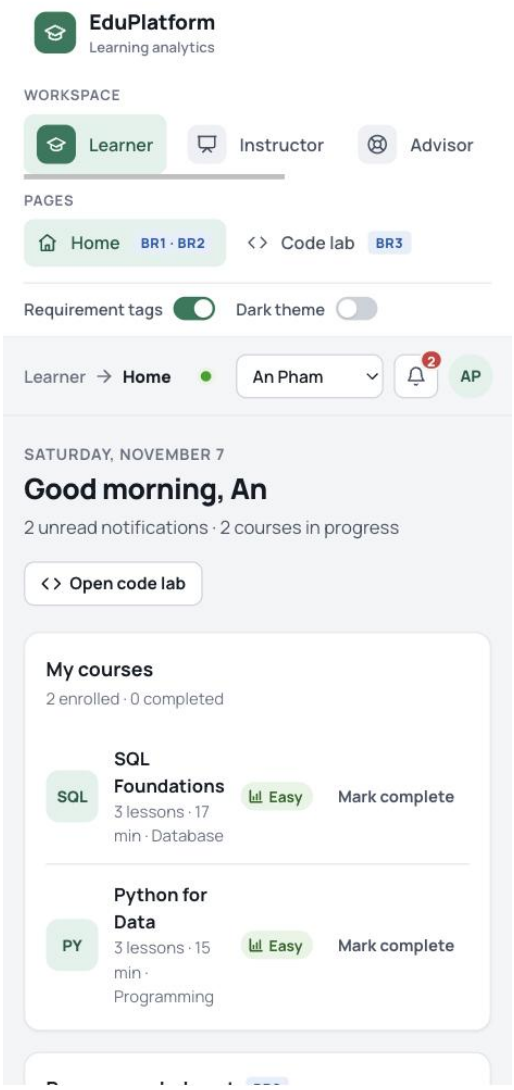
[TODO: thay bằng screenshot mới (hoặc giữ mockup có chủ đích)]

Hình 4.11. Mockup: Content & search (BR8), theo dõi một phiên bản từ khi xuất bản đến khi tìm kiếm được



[TODO: chụp lại ảnh mới]

Hình 4.12. Pipeline health (chế độ sáng): bộ đếm của từng chặng trong luồng dữ liệu



[TODO: chụp lại ảnh mới]

Hình 4.13. Trang người học trên màn hình điện thoại (390 px): thanh bên chuyển thành tab ngang

4.9.5 Kiểm thử giao diện

Giao diện được kiểm chứng bằng ba cách:

- **Biên dịch kiểu nghiêm ngặt**: TypeScript ở chế độ `strict`; bản build production thành công.
- **Kiểm tra thủ công trên trình duyệt** với dữ liệu thật ở chế độ sáng, tối, desktop và điện thoại. Một bài nộp thật đi qua sandbox và sinh ra một thông báo mới; các trang BR6–BR8 đọc mart thật từ Trino.
- **Kiểm thử chấp nhận bằng Playwright** (`apps/web/tests`), được cập nhật cho cấu trúc trang mới với các khẳng định nghiệp vụ giữ nguyên. Lần chạy sau khi thiết kế lại cho kết quả: ba test đạt (thông tin đăng nhập sai bị từ chối; báo cáo tổ chức ẩn nhóm nhỏ và chặn truy cập ngoài phạm vi; heatmap theo phiên bản và xu hướng tương tác). Các test còn lại

cần những dịch vụ không chạy vào thời điểm đó (container `br45`, OpenSearch, lớp phủ trực tiếp) nên chưa được chạy lại trên giao diện mới.

CHƯƠNG 5

KIỂM THỬ, ĐÁNH GIÁ VÀ ALTERNATIVE SOLUTIONS

Chương này trả lời hai câu hỏi: hệ thống có tính đúng không, và nhanh đến mức nào. Mọi số liệu dưới đây được đo trên dữ liệu tổng hợp, trên máy cá nhân với toàn bộ stack chạy cùng lúc. Chúng là quan sát cục bộ, không phải cam kết năng lực. Các file CSV gốc giữ mọi mẫu đo (Phụ lục A).

5.1 Chiến lược kiểm thử

Bảng 5.1. Các tầng kiểm thử

Tầng	Số lượng	Phạm vi
Đơn vị	346 **[TODO: 353 pass, 3 skip]**	Policy và use case của domain, phân quyền API với adapter giả, phân tích bản ghi Debezium, S3 (moto), Redis (fakeredis), OpenSearch adapter, lớp phủ, cache; test kiến trúc kiểm tra ranh giới các lớp
Spark	9 **[TODO: xác nhận]**	SQL của bộ đọc mart trên bảng Delta dựng từ `warehouse/ddl`; job reindex trên Delta với S3 giả lập; tính tương đương của lớp phủ streaming trên Kafka và Redis thật
Tích hợp BR4/BR5	13 **[TODO: 17 collected]**	PostgreSQL và API: SQL khớp Python, lần làm đầu/cuối, kết quả thiếu hoặc không hợp lệ, lỗi hạ tầng và lỗi mã, đổi phiên bản, LSN cũ, cách ly sự kiện trễ, phân quyền, vòng đời cảnh báo
Dịch vụ thật	–	OpenSearch 2.17 (analyzer, HNSW, lọc kNN, chỉ phiên bản hiện hành); danh mục PostgreSQL (5 người xuất bản đồng thời, rollback); tính tương đương lớp phủ kể cả gửi lại và phát lại toàn bộ
End-to-end	1 + 6	Luồng PostgreSQL → WAL → Debezium → Kafka → Spark → API với 6 kiểm tra chấp nhận, gồm sửa điểm, phát lại Kafka, UPDATE/DELETE hoạt động và khởi động lại checkpoint
Trình duyệt	7 **[TODO: 9 test]**	Playwright: drill-down của giảng viên và cập nhật khi sửa điểm, xác nhận cảnh báo và hồi phục rủi ro, lỗi đăng nhập, k-anonymity, heatmap theo phiên bản, xuất bản đến khi tìm kiếm được, lớp phủ trực tiếp

Ngoài ra, mã Python qua `ruff` và `mypy`; kho dữ liệu chạy cả 10 cổng chất lượng sau mỗi lần build.

5.2 Kiểm chứng nghiệp vụ

Mỗi BR có ít nhất một kịch bản dữ liệu được thiết kế để kiểm tra đúng quy tắc nghiệp vụ, rồi được quan sát đến tận giao diện.

Bảng 5.2. Các kịch bản kiểm chứng nghiệp vụ và kết quả

BR	Kịch bản	Kết quả quan sát
4	24 học viên, 3 test bắt buộc; 2 người đạt hết, 22 người trượt cùng một test	Câu hỏi đạt $2/24 = 8{,}33\%$; hai test đạt 100%, test lỗi đạt 8,33% và thấp hơn trung vị 91,67 điểm %: gắn cờ `SUSPICIOUS_TEST_CASE`. Sửa một kết quả trong PostgreSQL làm số hiển thị đổi từ 8% sang 13% mà không tải lại trang.
4	Kho dữ liệu: test hỏng `t5_pairs` và test khó nhưng hợp lệ `t3_ties`	Chỉ `t5_pairs` (0/68 lượt đạt) bị gắn cờ; `t3_ties` (0,55) không bị gắn cờ
5	Học viên có 4 hoạt động tuần trước, không hoạt động tuần này, một bài đến hạn chưa nộp	100 điểm, mức CRITICAL, sinh cảnh báo. Bốn hoạt động thật ghi qua API làm điểm giảm còn 50 và cảnh báo tự giải quyết. Phát lại Kafka không sinh cảnh báo trùng.
5	Kho dữ liệu: rủi ro theo ngày của một học viên	Một cảnh báo khi vượt ngưỡng; học viên bỏ học 6 ngày sau, nhãn kết quả `dropped_within_30d = true`
6	Globex có các nhóm 2 người; Acme có nhóm lớn	Globex chỉ công bố quy mô; Acme được báo cáo; truy cập ngoài phạm vi trả về 403
7	Video `v-sql-joins` được quay lại	Hai heatmap riêng: v1 (420 giây) và v2 (390 giây)
8	Xuất bản transcript có chuỗi đánh dấu duy nhất	Phiên bản mới tìm kiếm được, cache trả `miss` rồi được làm ấm; xoá qua CDC gỡ vector và mục tìm kiếm
–	Chạy kho dữ liệu: backfill, hai lần tăng dần, rồi chạy lại	Dấu vân tay các bảng giống hệt sau lần chạy lại (idempotent); một lần đồng bộ mobile trễ tự kéo cửa sổ gold lùi hai ngày

5.3 Môi trường đo

Bảng 5.3. Môi trường đo hiệu năng

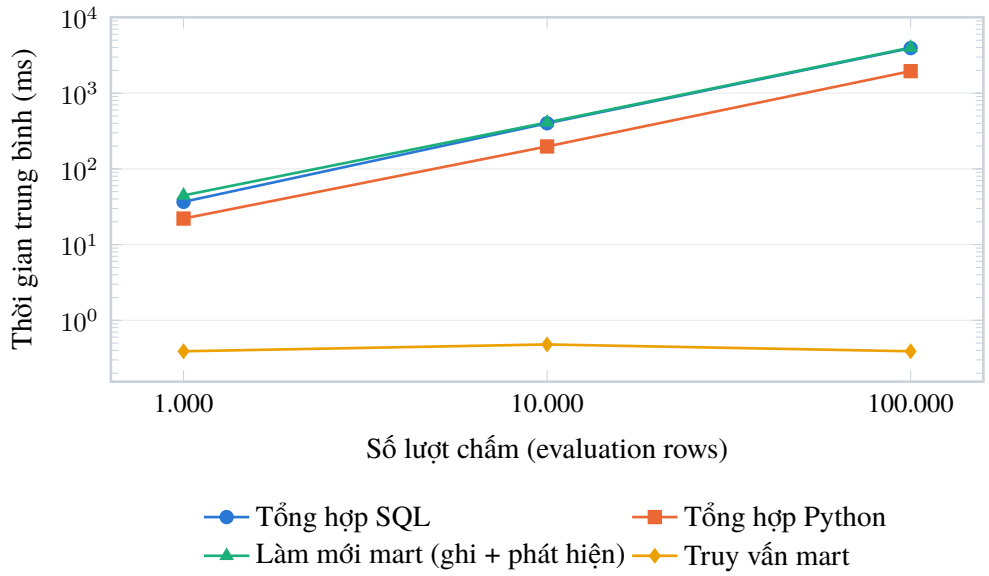
	BR4 / BR5 (16/10/2026)	**BR6–BR8 (02/11/2026)**
Máy	Linux x86-64, 6 CPU logic, khoảng 18,4 GiB RAM	macOS arm64, VM Colima 6 CPU, 10 GB
Engine	Spark 3.5.1, `local[2]`, driver 1 GiB; PostgreSQL 16.15	Spark 3.5.5, Delta 3.3.2; Trino 478 (heap 1,2 GB); OpenSearch 2.17.1
Lặp lại	3 lần mỗi kích thước	2–30 lần tuỳ thí nghiệm
Dữ liệu	Tổng hợp	Tổng hợp

5.4 Kết quả hiệu năng

5.4.1 BR4: tổng hợp SQL so với Python

[TODO: chạy lại benchmark BR4; cập nhật CSV + TikZ]

Cả hai cách dùng cùng các dữ kiện đã kiểm tra hợp lệ và cùng ngữ nghĩa chỉ số; kết quả khớp nhau ở mọi kích thước và mọi lần lặp. Hình 5.1 cho thấy thời gian tăng gần tuyến tính theo số lượt chấm. Python nhanh hơn khoảng hai lần vì dùng lại dữ kiện đã nối cho cả hai phép tổng hợp, còn SQL lặp lại một số phép nối JSONB. Nhóm vẫn giữ SQL cho đường phục vụ vì tính giao dịch và ngữ nghĩa rõ ràng của cơ sở dữ liệu. Truy vấn mart đã tổng hợp luôn dưới 0,5 ms.



Hình 5.1. BR4: thời gian tổng hợp và truy vấn theo quy mô (trung bình 3 lần, trục log)

5.4.2 BR5: độ trễ đầu cuối và thông lượng cửa sổ

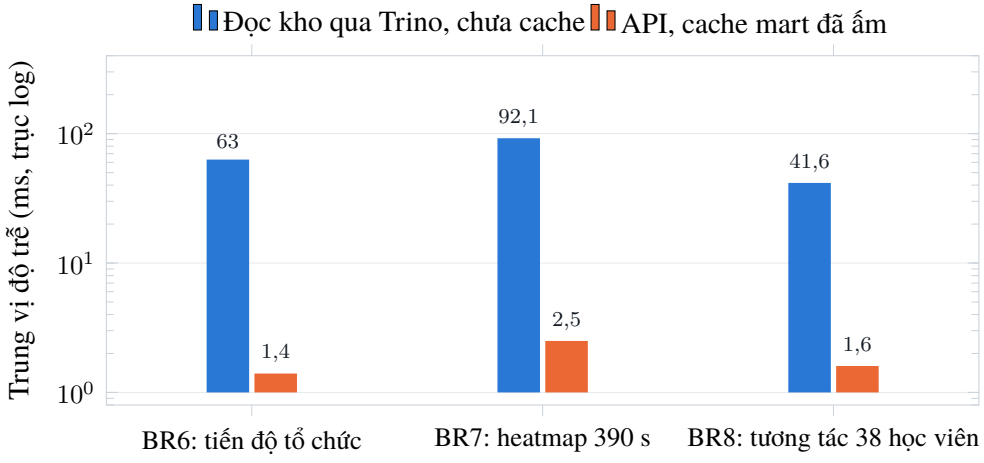
Lần chạy chấp nhận trên đường CDC thật đo được khoảng **10.164 ms** **[TODO: chạy lại BR5]** từ lúc ghi dữ liệu nguồn đến khi kết quả rủi ro và cảnh báo được lưu. Từ lúc khởi động lại Spark đến khi hồi phục mất khoảng **20.142 ms** **[TODO: chạy lại recovery]**, không có cảnh báo trùng sau phát lại. Thí nghiệm quy mô (Bảng 5.4) đo riêng chặng Kafka → Spark → cửa sổ trong PostgreSQL, với khoảng 1% bản ghi trùng; mọi kết quả khớp với một phép tổng hợp lô độc lập bằng Python.

Bảng 5.4. BR5: chặng Kafka → Spark → cửa sổ đã lưu (trung bình 3 lần) **[TODO: số mới]**

Sự kiện duy nhất	Kafka đến cửa sổ đã lưu	Thông lượng quan sát	Tính lô bằng Python
1.000	6.456 ms	167 sự kiện/s	4,4 ms
10.000	8.216 ms	1.332 sự kiện/s	29,0 ms
100.000	14.848 ms	6.742 sự kiện/s	296,8 ms

Phép tính lô rẻ hơn nhiều cho một ảnh chụp tĩnh. Streaming đổi lấy việc tiếp nhận liên tục, xử lý sửa đổi và phát lại, checkpoint và cập nhật tự động; một triển khai lô theo lịch sẽ còn phải chờ đến lượt chạy mới thấy thay đổi. Nhóm không đưa ra số liệu dương tính giả hay độ hiệu chỉnh của điểm rủi ro, vì không có nhãn bỏ học thật.

5.4.3 BR6–BR8: phục vụ mart

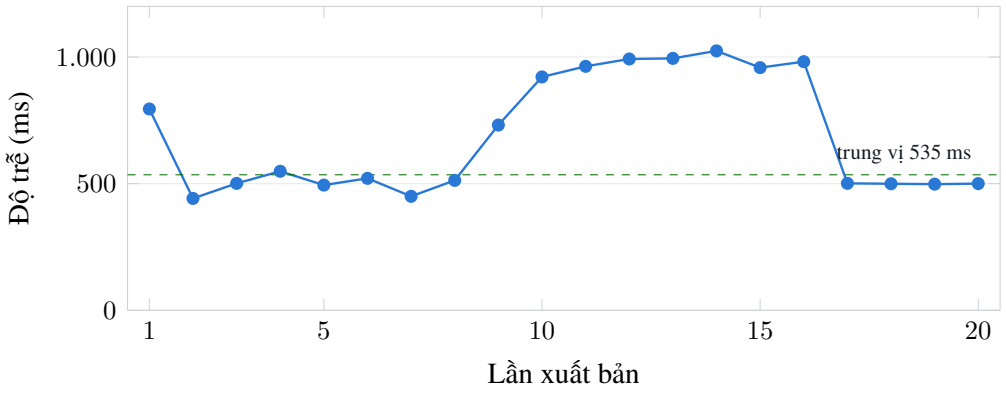


Hình 5.2. Độ trễ phục vụ các mart BR6–BR8 (30 request mỗi trường hợp) **[TODO: chạy lại serving]**

Bách phân vị 90 khi đọc thẳng qua Trino là 92, 136 và 59 ms cho BR6, BR7, BR8; khi có cache là 1,8, 2,8 và 2,0 ms. Vì mart chỉ thay đổi mỗi ngày còn dashboard thăm dò mỗi 5 giây, bộ đọc

có cache TTL 60 giây (`CachedMartReader`). Cache này còn sửa được một lỗi thật: Trino bị hết bộ nhớ khi dashboard thăm dò liên tục mà không có cache.

5.4.4 BR8: từ xuất bản đến khi tìm kiếm được



Hình 5.3. BR8: độ trễ từ xuất bản đến khi tìm kiếm được, 20 lần xuất bản liên tiếp **[TODO: chạy lại content sync]**

API nhận yêu cầu xuất bản trong 16,9 ms (trung vị). Toàn bộ đường đi PostgreSQL WAL → Debezium → Kafka → content-sync → tìm kiếm được có trung vị **535 ms** **[TODO: chạy lại BR8]**, p90 992 ms, lớn nhất 1.024 ms. Bản thân bước đồng bộ chỉ mất 25–70 ms; phần còn lại là thời gian bắt thay đổi và chu kỳ thăm dò của consumer.

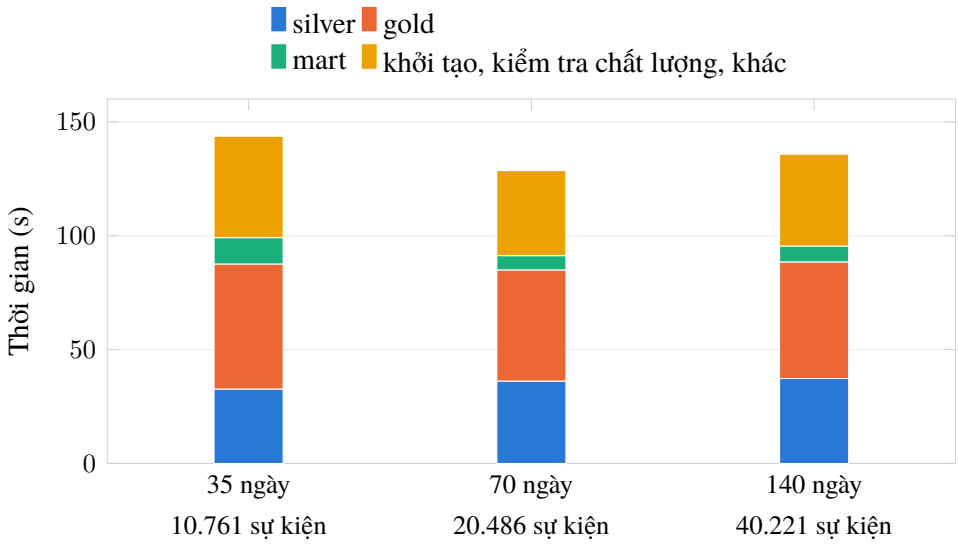
5.4.5 BR7/BR8: lớp phủ thời gian thực

Bảng 5.5. Độ trễ từ sự kiện đến khi hiển thị trong API (3 đợt mỗi kích thước) **[TODO: chạy lại live overlay]**

Phiên mỗi đợt	Sự kiện thô (trung bình)	Trung bình (nhỏ nhất – lớn nhất)	Chính xác
25	234	7,27 s (7,19 – 7,35)	có
100	888	7,23 s (6,95 – 7,38)	có
400	3.622	7,12 s (5,69 – 8,03)	có

Độ trễ gần như không đổi khi tải tăng 15 lần, vì nó bị chi phối bởi chi phí cố định của mỗi micro-batch (bước chuẩn hoá khoảng 2,7 s, trigger 1 s) chứ không phải khối lượng. “Chính xác” nghĩa là lớp phủ chứa đúng số tương tác đúng hạn kỳ vọng của đợt cho video nhiều tương tác nhất.

5.4.6 Thời gian build kho dữ liệu



Hình 5.4. Thời gian build toàn bộ 43 model theo khối lượng dữ liệu (trung bình 2 lần) **[TODO: chạy lại 35/70/140 ngày]**

Tổng thời gian build là 143,7, 128,6 và 135,8 giây **[TODO: số mới]** cho 35, 70 và 140 ngày dữ liệu; mọi model đều thành công. Thời gian gần như phẳng khi khối lượng tăng 4 lần: ở quy mô này, chi phí cố định của mỗi model (khởi tạo job Spark, commit Delta, kiểm tra chất lượng) chiếm phần lớn. Các mart BR6–BR8 chỉ mất 1–6 giây. Chênh lệch giữa hai lần chạy (123 và 164 giây ở 35 ngày) là nhiễu do máy ảo dùng chung với phần còn lại của stack.

5.5 Thảo luận

- **Đáp ứng yêu cầu độ trễ.** Cảnh báo BR5 khoảng 10 giây và tìm kiếm BR8 khoảng nửa giây đều đủ cho mục đích nghiệp vụ (cố vấn can thiệp, người học tìm thấy nội dung mới). Lớp phủ khoảng 7 giây là chấp nhận được cho dashboard trên mart theo ngày, nhưng sẽ cần Flink nếu muốn dưới một giây (ADR 0014).
- **Tách biệt tính toán và phục vụ.** Mart tổng hợp sẵn cộng cache cho độ trễ phục vụ vài mili-giây dù truy vấn gốc qua Trino mất hàng chục mili-giây; tra cứu theo khoá vẫn ở Redis.
- **Đúng đắn quan trọng hơn tốc độ.** Ở BR4, nhóm giữ SQL dù Python nhanh hơn, và ghi rõ kết quả benchmark thay vì tuyên bố SQL thắng. Hướng tối ưu (bảng sự kiện có kiểu, dùng chung kết quả trung gian) được ghi vào hạn chế.
- **Lỗi tìm ra nhờ kiểm chứng.** Quá trình đo và kiểm thử đã phát hiện và sửa nhiều lỗi thật. Ví dụ: bộ chia đoạn bỏ mất khoảng 2/3 transcript; đồng hồ thiết bị chạy nhanh làm rơi sự

kiện đúng hạn của người khác; cache tài liệu bị đọc trước khi kiểm tra phạm vi; Trino hết bộ nhớ khi dashboard thăm dò liên tục.

5.6 Alternative solutions

Mục này so sánh stack hiện tại với các phương án tốt hơn khi dữ liệu lớn. Các số đo được chạy trên cùng một máy, cùng bộ dữ liệu sinh 5 GiB, không dùng số liệu suy diễn. Do giới hạn RAM của máy (7,4 GiB), các phép đo dùng các tiền tố 1M/5M/10M dòng từ file 5 GiB; cùng seed và cùng schema. Kết quả thô nằm trong `benchmarks/alternatives/results/`.

5.6.1 Dữ liệu và phương pháp

Bảng 5.6. Dataset 5 GiB dùng cho benchmark alternative

Cặp	File	Rows	Bytes
Streaming	`events-5gb.jsonl`	17.410.669	5.368.709.155
OLAP	`org-facts-5gb.csv`	44.492.023	5.368.709.188
Feature store	`features-5gb.jsonl`	14.375.660	5.368.709.338

Mỗi phép đo có một lần warm-up và ba lần lặp; báo cáo trung vị p50. Với OLAP, hai engine nhận cùng file CSV và chạy cùng một câu SQL tổng hợp theo `(org_id, course_id)`. Với feature store, hai engine nhận cùng JSONL và thực hiện cùng một batch khoảng 10.000 lookup theo khoá.

5.6.2 OLAP: PostgreSQL và ClickHouse

StarRocks không được cài trên máy này; ClickHouse là fallback MPP/columnar theo kế hoạch. Bảng 5.7 so sánh PostgreSQL (row store, hiện tại) với ClickHouse (column store, alternative).

Bảng 5.7. OLAP: load và query p50 khi số dòng tăng

Rows	PG load	CH load	PG p50	CH p50	Speed-up
1.000.000	10,80 s	4,43 s	6.289 ms	84,96 ms	74,0×
5.000.000	53,44 s	10,41 s	33.829 ms	369,81 ms	91,5×
10.000.000	96,24 s	17,79 s	69.898 ms	664,76 ms	105,1×

Cả 12 nhóm `(org_id, course_id)` khớp giữa hai engine; count khớp tuyệt đối, trung bình lệch dưới 10^{-9}. PostgreSQL phù hợp với mart nhỏ và cập nhật giao dịch; ClickHouse thắng rõ từ 1M dòng và khoảng cách tăng theo quy mô. Đây là lý do alternative columnar đáng chuyển khi mart lớn, dù vận hành thêm một hệ thống.

5.6.3 Feature store: Redis và Memcached

Hai engine cùng lớp in-memory KV nhận cùng 1M/5M key, cùng payload JSON, cùng 10 batch 1.000 key. Cả hai chạy với 4 GB, không evict; mỗi bên có một lần warm-up và ba lần lặp.

Bảng 5.8. Feature store: Redis và Memcached, cùng batch 1.000 key

Keys	Redis load	Memcached load	Redis p50/batch	Memcached p50/batch	Hit
1.000.000	23,10 s	14,72 s	6,93 ms	6,66 ms	100%
5.000.000	121,79 s	73,84 s	8,13 ms	6,68 ms	100%

Ở 5M, Redis dùng 2,38 GB và Memcached dùng 2,36 GB; cả hai giữ đủ 100% key. Memcached nhanh hơn khoảng 4% ở 1M và 18% ở 5M cho lookup, và nạp nhanh hơn đáng kể. Đây là so sánh cùng setup: cùng dữ liệu, cùng batch, cùng số round trip.

PostgreSQL được giữ làm reference cho store bền/giao dịch, không phải đối thủ tốc độ: cùng 5M key, load 99,43 s và p50 66,54 ms/batch. Nó chậm hơn cả Redis và Memcached cho lookup, nhưng giữ được giao dịch và là nguồn sự thật.

Memory pressure. Khi Redis bị cap 512 MB, 5M key vượt dung lượng: Redis evict 3.926.686 key, chỉ còn 21,68% key và p50 147,9 ms. Con số đó không dùng để so trực tiếp vì 78% request là miss; nó chỉ chứng minh Redis cần đủ RAM hoặc scale-out.

Kết luận. Memcached nhanh hơn Redis cho pure KV lookup ở cả 1M và 5M. Redis vẫn có lợi thế về cấu trúc dữ liệu, TTL, pub/sub và persistence; PostgreSQL phù hợp khi cần giao dịch/nguồn sự thật, không phải cache.

5.6.4 Streaming: Spark Structured Streaming và Flink

Hai engine đọc cùng topic Kafka `bench.events3` với 100.000 event, lọc `score > 0.5` và ghi kết quả sang topic đích. Bảng 5.9 đo thời gian xử lý của job (không tính thời gian khởi động SQL client/JAR).

Bảng 5.9. Streaming filter: cùng input, cùng output

Engine	Job runtime	Submit→output	Input ev/s	Output ev/s
Flink 1.19 SQL	9,16 s	22 s	10.917	5.449
Spark 3.5 Structured Streaming	9,0 s	44 s	11.111	5.546

Hai bên ghi đúng 49.913 event; kiểm tra theo `event_id` cho kết quả `missing_in_spark = 0`, `missing_in_flink = 0`. Chênh lệch `submit$\to$output` chủ yếu do thời gian khởi động client và tải dependency, không phải tốc độ xử lý. Bảng này đo pipeline filter; cửa sổ cảnh báo 7 ngày là một thí nghiệm SLA riêng, không dùng các số filter này để đại diện.

5.6.5 Khi nào chuyển stack

Bảng 5.10. Điều kiện chuyển sang alternative

Vai trò	Alternative	Điều kiện chuyển
OLAP mart	ClickHouse/StarRocks	Mart vượt vài triệu dòng, query p95 > 1–2 s hoặc nhiều dashboard đồng thời
Feature serving	Memcached/Redis Cluster	Pure KV throughput cao hơn; cần thêm TTL/persistence thì giữ Redis
Streaming	Flink	Cần p95 < 1 s, state lớn hoặc checkpoint Spark không đáp ứng SLA

Các kết quả này là quan sát cục bộ trên một máy, không phải cam kết năng lực production.

CHƯƠNG 6

HẠN CHẾ, KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Nhóm liệt kê các hạn chế một cách trung thực, để người đọc phân biệt phần đã chạy thật với phần còn ở mức demo hoặc thiết kế.

6.1 Hạn chế hiện tại

1. **BR1–BR3 đã có đường dịch vụ, nhưng sandbox và provider vẫn là mức demo.** Dịch vụ thông báo và dịch vụ chấm bài chạy Kafka/Redis, API đọc mô hình Redis và chuyển bài nộp qua BFF; BR3 ghi schema riêng `grading.*`, còn projector sang LMS `public.*` là tùy chọn. Sandbox vẫn là tiến trình con và SQLite cục bộ, không phải ranh giới bảo mật; email/push cần thông tin xác thực provider thật để dùng ngoài local.
2. **BR2 chưa có mô hình học máy.** Cả hai nhánh đặc trưng đã được hiện thực và phục vụ qua Redis (bảng lô được materialise, phiên chạy streaming), nhưng thứ hạng vẫn do luật quyết định; chưa có mô hình học từ dữ liệu hay bộ nhãn thật để so sánh.
3. **Độ trễ streaming ở mức giây.** Spark Structured Streaming cho cảnh báo BR5 sau khoảng 10 giây và lớp phủ sau khoảng 7 giây, cao hơn mức dưới một giây mà Flink có thể đạt. Đây là đánh đổi có chủ đích (ADR 0009, 0014).
4. **Embedding mang tính từ vựng.** Bộ nhúng băm không hiểu ngữ nghĩa ("join" không khớp "merge"). Đã có cổng `Embedder` và đường reindex để thay bằng mô hình thật.
5. **Lớp phủ không mang phiên bản nội dung.** Vào ngày một video được quay lại, lớp phủ trộn hai phiên bản; vì vậy nó chỉ được hiển thị cho phiên bản mới nhất.
6. **Lịch sử BR5 trong kho dữ liệu đến từ dữ liệu tổng hợp.** Runtime BR4/BR5 lưu điểm và cảnh báo trong PostgreSQL; cần phát chúng lên `risk.learner_score.v1` và `risk.dropout_alert.v1` (hoặc thêm bảng vào connector CDC) để kho dữ liệu nhận từ đường thật.
7. **Hiệu năng BR4.** Mỗi lần sửa dữ liệu làm mới toàn bộ bản tổng hợp bị ảnh hưởng và dùng phép nối JSONB; với 100.000 lượt chấm, một lần sửa tốn khoảng 4 giây. **[TODO: xác nhận lại BR4]**
8. **Vận hành đơn nút, bảo mật cục bộ.** Trino, OpenSearch, Kafka chạy một nút; plugin bảo mật của OpenSearch tắt; token tĩnh thay cho hệ thống định danh của trường. Chưa có giám sát (độ trễ consumer, thời gian checkpoint, SLA độ tươi).

9. **Dữ liệu tổng hợp, không nhãn thật.** Không có nhãn bỏ học thật nên không đánh giá được độ chính xác của cảnh báo; các số đo hiệu năng là quan sát trên một máy.
10. **Lệch dữ liệu mẫu.** Seed của lakehouse khai báo khoảng 18 khoá học và video không có trong cơ sở dữ liệu LMS mẫu, nên job reindex báo các mục này là lỗi.

6.2 Hướng phát triển

- Đưa projector BR3 vào CI/e2e và thử bật đồng bộ LMS trên một bản sao dữ liệu, kèm quy trình re-baseline báo cáo.
- Thử một mô hình gợi ý học từ dữ liệu cho BR2, so sánh với luật hiện tại; hai nhánh đặc trưng đã có trong Redis.
- Chuyển các job cần độ trễ dưới một giây sang Flink khi nền tảng lên Flink 2.x và connector Delta Kernel ổn định.
- Thay bộ nhúng băm bằng mô hình sentence-embedding, chạy một lần reindex bắt buộc.
- Chuyển các bảng snapshot (`fct_enrollment`, `fct_lesson_progress`) sang MERGE dựa trên Change Data Feed khi khối lượng đủ lớn; với dashboard dưới giây ở quy mô lớn, gắn StarRocks vào cùng Hive Metastore.
- Triển khai lên Kubernetes với nhiều nút, TLS và xác thực tập trung; giám sát bằng Prometheus cùng Grafana; bổ sung quy trình xoay khoá bút danh.
- Thu thập nhãn kết quả thật để hiệu chỉnh và đánh giá điểm rủi ro BR5.

6.3 Kết luận

Báo cáo đã trình bày DE_261, một nền tảng kỹ thuật dữ liệu biến bốn nguồn dữ liệu rời rạc của một hệ thống giáo dục trực tuyến thành các kho dữ liệu phục vụ tám yêu cầu nghiệp vụ.

Về kiến trúc, nền tảng bám sát kiến trúc tham chiếu của đề bài: Kafka và Debezium cho tiếp nhận, Spark cho xử lý, Delta Lake cho lakehouse, Redis cho phục vụ độ trễ thấp. Lớp gold được mô hình hoá thành kho dữ liệu Kimball và phục vụ qua Trino. Các điểm điều chỉnh, quan trọng nhất là dùng Spark Structured Streaming thay Flink cho BR5 và các lớp phủ, đều được ghi lại bằng ADR kèm lý do và hệ quả. Tổ chức mã theo DDD và kiến trúc lục giác cho phép cùng một logic nghiệp vụ chạy trên nhiều engine và được kiểm thử độc lập.

Về hiện thực, BR4–BR8 chạy trên hạ tầng thật với các tính chất mà một hệ thống dữ liệu cần có: xử lý theo thời gian sự kiện, chịu được dữ liệu trễ, trùng lặp và sai thứ tự, chạy lại idempotent, khử định danh ở ranh giới silver, k-anonymity cho báo cáo tổ chức, và cổng kiểm tra chất lượng sau mỗi lớp. BR1–BR3 có logic nghiệp vụ hoàn chỉnh và chạy trong runtime demo. Một giao

diện web mới đưa kết quả đến đúng từng nhóm người dùng.

Về đánh giá, hệ thống có hơn 360 test tự động ở nhiều tầng và các kịch bản kiểm chứng nghiệp vụ được quan sát đến tận giao diện. Các số đo cho thấy độ trễ phù hợp với từng yêu cầu: cảnh báo bỏ học khoảng 10 giây sau thay đổi nguồn, nội dung mới tìm kiếm được sau khoảng nửa giây, mart được phục vụ trong vài mili-giây khi có cache.

Bài học lớn nhất của nhóm là giá trị của việc kiểm chứng bằng dữ liệu và số đo. Nhiều lỗi quan trọng chỉ lộ ra khi chạy hệ thống thật và đo đạc. Việc ghi rõ phần nào đã chạy, phần nào còn là thiết kế giúp nhóm và người đọc đánh giá đúng mức độ hoàn thiện, và định hướng cho các bước phát triển tiếp theo.

TÀI LIỆU THAM KHẢO

[1] Martin Kleppmann. *Designing Data-Intensive Applications*. O'Reilly Media, 2017.

[2] Advanced Distributed Learning Initiative. Experience API (xAPI) Specification. `https://github.com/adlnet/xAPI-Spec`, 2017.

[3] Jay Kreps, Neha Narkhede, and Jun Rao. Kafka: a Distributed Messaging System for Log Processing. In *Proceedings of the NetDB Workshop*, 2011.

[4] Debezium Community. Debezium: Stream changes from your database. `https://debezium.io/documentation/`, 2026. Truy cập 11/2026.

[5] Matei Zaharia, Reynold S. Xin, Patrick Wendell, et al. Apache Spark: A Unified Engine for Big Data Processing. *Communications of the ACM*, 59(11):56–65, 2016.

[6] Michael Armbrust, Tathagata Das, Joseph Torres, et al. Structured Streaming: A Declarative API for Real-Time Applications in Apache Spark. In *Proceedings of the 2018 International Conference on Management of Data (SIGMOD)*, pages 601–613, 2018.

[7] Michael Armbrust, Tathagata Das, Liwen Sun, et al. Delta Lake: High-Performance ACID Table Storage over Cloud Object Stores. *Proceedings of the VLDB Endowment*, 13(12):3411–3424, 2020.

[8] Raghav Sethi, Martin Traverso, Dain Sundstrom, et al. Presto: SQL on Everything. In *IEEE 35th International Conference on Data Engineering (ICDE)*, pages 1802–1813, 2019.

[9] Redis Ltd. Redis Documentation. `https://redis.io/docs/`, 2026. Truy cập 11/2026.

[10] OpenSearch Project. OpenSearch Documentation: k-NN and hybrid search. `https://opensearch.org/docs/latest/`, 2026. Truy cập 11/2026.

[11] Eric Evans. *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley, 2003.

[12] Alistair Cockburn. Hexagonal Architecture (Ports and Adapters). `https://alistair.cockburn.us/hexagonal-architecture/`, 2005.

[13] Tyler Akidau, Robert Bradshaw, Craig Chambers, et al. The Dataflow Model: A Practical Approach to Balancing Correctness, Latency, and Cost in Massive-Scale, Unbounded, Out-of-Order Data Processing. *Proceedings of the VLDB Endowment*, 8(12):1792–1803, 2015.

[14] Michael Nygard. Documenting Architecture Decisions. `https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions`, 2011.

[15] Paris Carbone, Asterios Katsifodimos, Stephan Ewen, Volker Markl, Seif Haridi, and Kostas Tzoumas. Apache Flink: Stream and Batch Processing in a Single Engine. *Bulletin of the IEEE Computer Society Technical Committee on Data Engineering*, 38(4), 2015.

[16] Michael Armbrust, Ali Ghodsi, Reynold Xin, and Matei Zaharia. Lakehouse: A New Generation of Open Platforms that Unify Data Warehousing and Advanced Analytics. In *Conference on Innovative Data Systems Research (CIDR)*, 2021.

[17] Databricks. What is the medallion lakehouse architecture? `https://docs.databricks.com/en/lakehouse/medallion.html`, 2026. Truy cập 11/2026.

[18] Ralph Kimball and Margy Ross. *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*. Wiley, 3 edition, 2013.

[19] Latanya Sweeney. k-Anonymity: A Model for Protecting Privacy. *International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems*, 10(5):557–570, 2002.

[20] Yu. A. Malkov and D. A. Yashunin. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4):824–836, 2020.

[21] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Buettcher. Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods. In *Proceedings of the 32nd International ACM SIGIR Conference*, pages 758–759, 2009.

PHỤ LỤC A

HƯỚNG DẪN CHẠY VÀ TÁI LẬP KẾT QUẢ

A.1 Yêu cầu

Docker (khuyến nghị 8 GB RAM cho VM), Python 3.11 trở lên, Node.js 20 trở lên. Postgres của dự án mặc định ở cổng 5433 để tránh xung đột với các dự án khác.

A.2 Khởi động

BR1–BR3.

```
docker compose -f docker-compose.yml -f docker-compose.br13.yml \
  up -d --build kafka-init br13-init grading-ingest grading-worker \
  grading-normalizer notification demo mailpit push-sink


curl -s http://localhost:8000/api/health   # {"status":"ok","mode":"service"}


docker compose -f docker-compose.yml -f docker-compose.br13.yml \
  --profile jobs run --rm spark-jobs edu batch features-online --run-date
    2026-11-10


docker compose -f docker-compose.yml -f docker-compose.br13.yml \
  --profile br13-project up -d grading-projector
```

BR4/BR5.

```
docker compose -f docker-compose.yml -f docker-compose.br13.yml \
  --profile br45 up -d --build demo br45-init br45-cdc br45-stream
docker compose exec -T br45-cdc python -m edu_platform.br45.cli seed
```

BR6/BR7/BR8. :

```
docker compose -f docker-compose.yml -f docker-compose.br13.yml \
  --profile br678 up -d --build       # first run takes about 4-5 minutes
docker compose exec demo edu telemetry simulate --sessions 50
```

Kho dữ liệu với dữ liệu tổng hợp:

```
docker compose --profile jobs run --rm spark-jobs edu warehouse bootstrap
docker compose --profile jobs run --rm spark-jobs edu warehouse seed-bronze --
    end-date 2026-10-17 --days 35
docker compose --profile jobs run --rm spark-jobs edu warehouse run --run-date
     2026-10-17 --start-date 2026-08-01
docker exec -it de261-trino trino --user analyst --catalog lakehouse
```

Mở ứng dụng tại `http://localhost:8000`. Token demo cục bộ: `instructor-demo`, `advisor-demo`, `admin-demo` (cả hai tổ chức), `acme-admin-demo` (chỉ Acme).

A.3 Kiểm thử

```
.venv/bin/ruff check src tests && .venv/bin/mypy
.venv/bin/python -m pytest tests/unit                          # 346 unit tests
    # TODO: 353 pass, 3 skip
.venv/bin/python -m pytest -m spark tests/integration          # requires JDK
EDU_BR45_E2E=1 .venv/bin/python -m pytest tests/e2e/test_br45.py
cd apps/web && npm ci && npm run build && npx playwright install chromium
BR45_BASE_URL=http://localhost:8000 npx playwright test
BR45_BASE_URL=http://localhost:8000 npx playwright test tests/br123.spec.ts
    # BR1-BR3
```

A.4 Đo hiệu năng

```
.venv/bin/python -m edu_platform.br45.cli benchmark            # BR4
.venv/bin/python -m edu_platform.br45.stream_benchmark         # BR5
uv run edu benchmark serving --out docs/evidence/br678/serving_latency.csv
uv run edu benchmark content --out docs/evidence/br678/content_sync_latency.
    csv
uv run edu benchmark live    --out docs/evidence/br678/live_overlay.csv
```

Các file CSV gốc dùng trong Chương 5 được chép vào thư mục `report/data/`: `br4_benchmark.csv`, `br5_stream_benchmark.csv`, `serving_latency.csv`, `content_sync_latency.csv`, `live_overlay.csv`, `mart_build.csv`.

A.5 Đo alternative solutions

Các benchmark alternative dùng file 5 GiB trong `benchmarks/alternatives/data/` (gitignored). Chạy trên các tiền tố 1M/5M/10M dòng:

```
# OLAP: PostgreSQL vs ClickHouse (profile bench-olap)
docker compose -f docker-compose.yml -f docker-compose.bench.yml \
  --profile bench-olap up -d bench-clickhouse
.venv/bin/python -m benchmarks.alternatives.olap.load_postgres \
  benchmarks/alternatives/data/org-facts-1m.csv
.venv/bin/python -m benchmarks.alternatives.olap.load_clickhouse \
  benchmarks/alternatives/data/org-facts-1m.csv
.venv/bin/python -m benchmarks.alternatives.olap.query_olap \
  --engine both --repeat 3 --label 1m

# Feature store: Redis ịb ógii ạhn 512 MiB vs PostgreSQL disk
docker compose -f docker-compose.yml -f docker-compose.bench.yml \
  --profile bench-kv up -d bench-redis
.venv/bin/python -m benchmarks.alternatives.feature_store.load_redis \
  benchmarks/alternatives/data/features-1m.jsonl
.venv/bin/python -m benchmarks.alternatives.feature_store.load_postgres \
  benchmarks/alternatives/data/features-1m.jsonl
.venv/bin/python -m benchmarks.alternatives.feature_store.workload \
  --rows 1000000 --ops 10000 --label 1m
```

Kết quả chi tiết: `benchmarks/alternatives/results/`.

A.6 Biên dịch báo cáo

```
cd report
tectonic main.tex            # or: latexmk -xelatex main.tex
```

Trên Overleaf, chọn trình biên dịch XeLaTeX.

PHỤ LỤC B

TÓM TẮT API

Tất cả endpoint có tiền tố `/api`. Tài liệu OpenAPI đầy đủ ở `http://localhost:8000/docs`; hợp đồng chi tiết ở `docs/api.md`. Các endpoint phân tích yêu cầu token Bearer; sai hoặc thiếu token trả 401, sai vai trò hoặc ngoài phạm vi trả 403, dữ liệu không có trả 404, truy vấn sai trả 422, kho không sẵn sàng trả 503.

Bảng B.1. Các endpoint chính

BR	Phương thức	Đường dẫn	Mục đích
1	GET	`/learners/{id}/notifications`	Thông báo của người học
2	GET	`/learners/{id}/recommendations`	Gợi ý và khoá đã ghi danh
3	POST	`/submissions`	Nộp bài (202, xếp hàng)
3	GET	`/submissions/{id}`	Kết quả và phản hồi
4	GET	`/instructor/courses/{course}/questions/statistics`	Thống kê câu hỏi
4	GET	`/instructor/questions/{question}/test-cases`	Thống kê test case
5	GET	`/advisor/courses/{course}/at-risk-students`	Hàng đợi rủi ro, tóm tắt mức
5	GET	`/advisor/students/{student}/risk-history`	Lịch sử điểm
5	GET	`/advisor/alerts`	Danh sách cảnh báo
5	PATCH	`/advisor/alerts/{id}/acknowledge`, `/resolve`	Vòng đời cảnh báo
6	GET	`/admin/organisations/{org}/skill-progress`	Báo cáo tổ chức (k = 5)
7	GET	`/instructor/courses/{course}/videos`	Video và phiên bản
7	GET	`/instructor/videos/{id}/interaction-heatmap`	Heatmap kèm lớp phủ

BR	Phương thức	Đường dẫn	Mục đích
8	GET	`/instructor/courses/{course}/engagement-trend`	Xu hướng tương tác
8	POST	`/instructor/content/{id}/versions`	Xuất bản phiên bản (202)
8	GET	`/instructor/courses/{course}/search`	Tìm kiếm lai hoặc từ khoá
–	GET	`/events`	Server-Sent Events cho sự kiện trực tiếp